

Once, and only effectively: idempotency keys, heartbeats, replay, a Redis you can lose, progress and rate limits

The win: you can explain, to an interviewer or to yourself, why "exactly once" is never a property of a delivery and how it is manufactured anyway; why an idempotency key is claimed with a primary key inside the job's own transaction and what twenty simultaneous retries do while one of them holds the row; why a replay answers from the job as it now is and a refused run gives its key back; how a heartbeat differs from a lease, why a timeout cannot tell slow from dead, and what the attempt number in every `UPDATE ... WHERE` does about that; why an `op id` is a hash of the job, the step *and the op's content*; why a Redis flush changed no code and what a chaos check must refuse to call a pass; why a run's progress is a fenced column read by a poll that already existed; why the rate limit is one upsert in Postgres inside the run's transaction with `Retry-After` from the database's clock and no way to fail open; how fixed windows, sliding windows and token buckets differ and when each is right; which `X-Forwarded-For` entry is the client and whose word you took for it; where all of this sits against Stripe, Sidekiq, Celery, Spring Batch and Kafka; and you can point at the line of code for every claim.

Verification status, honestly. Every commit in this chapter passed lint, the type checker and the unit tests (`replay.test`, the worker's and the web's `progress.test`, `limit.test`, `usage.test`, `client-address.test`, `http-limits.test` against an in-memory limiter, `rebuild.test` over the chaos judge, the contracts, the config, the route table), and the guards named in the commits were mutation-checked. The integration suites (`api idempotency.int` with twenty simultaneous starts, `worker crash.int` including the Redis wipe, `db job-heartbeat.int`, `worker starvation.int`, `db run-progress.int`, `api rate-limit.int` over two instances, `db usage.int`, `api http-rate-limit.int` with the mint limit), the two end-to-end specs (`progress`, `usage-view`) and the two chaos scripts (`kill-worker-resumes`, `redis-wipe-rebuild`) are written and committed and *have not been run*: the host was too loaded to run Docker suites beside other work. They run on `build/epics` before anything merges to `main`. Where this page says "measured", it was measured by a unit test that ran; where it says "the test asserts", the test exists and is unrun. Section 10 is the by-hand demo to run first.

How to use this page. Every code block is cut from the repo at commit `1c9db7c`, so what you read is what is committed. Sections 1, 2 and 3 are the ones a distributed-systems interview will dig into; section 6 is the one a "design a rate limiter" question turns on; section 7 is the one a security review asks about; section 8 is the comparison you will be asked to make.

```
./init.sh # migrations 0018 to
0020: keys, heartbeats, limits
pnpm exec vitest run apps/worker/src/replay.test.ts packages/db/src/limit.test.ts apps/api/src/client-
address.test.ts apps/api/src/http-limits.test.ts # the pure halves
docker compose kill -s SIGKILL worker # mid-run;
watch the row, then start it again
```

Try it, once the stack is up: section 10 walks the double press, the kill, the flush, the reload and the limits, then the suites, with the exact commands and what each should show.

The map: what a job is promised here, and who keeps the promise

- 1 · Idempotency keys: the claim is the row, and the answer is the job as it now is
- 2 · Heartbeats, the stale sweep, and the attempt as a fence
- 3 · Replaying a dead attempt without a duplicate: ids that are functions of the step
- 4 · Redis is a cache: the wipe, and why no code changed

- 5 · Progress through the row: steps that survive a reload and never show a dead attempt's work
- 6 · Rate limits in Postgres: one upsert, inside the transaction, with the database's clock
- 7 · Every route limited, and who is asking: per user, per address, and the header you may not believe
- 8 · Trade-offs and comparisons: where this design sits
- 9 · Testing a crash you cannot schedule, a race you can, and a limit to the second
- 10 · By hand: press twice, kill the worker, flush Redis, reload mid-run, hit the limit
- 11 · What the type system could not see, epic 9 edition

Your turn: three drills

Recall, then test yourself

The map: what a job is promised here, and who keeps the promise

Epic 3 made a job a row: the `jobs` table is the run, the preview and the ship, and the queue only tells a worker to look (SPEC §2.9). Epics 4 to 8 built on that without asking what happens when the parts around the row misbehave: a browser that presses twice, a worker that dies with a job half done, a Redis that is flushed, a page reloaded mid-run, a tenant that asks for more than its share. Epic 9 asks those five questions and answers each one from the same place, the row in Postgres. **A press is one job** (F27): a retry with the same `Idempotency-Key` is answered with the job the first press made, however many arrive at once, because the key is claimed with the row's primary key before the job exists. **A dead worker's job runs again, once** (F28): a running job heartbeats its row every second, a sweep puts a silent one back in the queue, and the attempt number fences a worker that was only slow. **Losing Redis loses nothing** (F28, §4): the sweep rebuilds the queue from whatever Postgres still calls `queued`, and no code changed to make that true, only the proof was missing. **Progress is a column** (F30): the run's steps ride the row, written by the attempt that holds it, read by whoever asks. **Limits are counted where the work is** (F31): one upsert in the run's own transaction for AI runs, and the same upsert for every HTTP route, so every api instance shares the count and nothing fails open.

STEP	WHERE	AND	WHAT HAPPENS
1	packages/db/migrations/0018_idempotency_keys.sql	packages/db/src/index.ts	<code>idempotency_keys</code> with <code>primary key (org_id, user_id, key)</code> ; withKey <code>claims</code> with <code>insert ... on conflict do nothing</code> in a transaction, makes the job, records its id; the loser waits on the row and answers from the job's row as it now is; 422 for the same key with another request
2	packages/db/migrations/0019_job_heartbeat.sql	apps/worker/src/worker.ts	<code>heartbeat_at</code> and <code>attempts</code> ; <code>claim()</code> is the next attempt; a 1 s beat that also answers cancel; <code>requeueStale</code> in one statement with <code>for update skip locked</code> ; <code>finish</code> and <code>report fenced</code> by the attempt
3	apps/worker/src/ai.ts	apps/worker/src/tools.ts	<code>replayIds</code> : the n-th node id and the n-th op id are functions of (job, n, the op); <code>add_node</code> on a node that already exists with the same shape is <i>done</i>, not refused; the room's (sender, opId) dedupe from E6.1 answers a replayed op with its original seq
4	scripts/chaos/redis-wipe-rebuild.ts	scripts/chaos/rebuild.ts	no production change; two rounds (FLUSHALL, restart) under running and waiting runs and an open room; the judge asks: every run succeeded with <code>attempts = 1</code> , every step journaled once, the lease back under a larger token that the journal's fence names
5	apps/worker/src/progress.ts	apps/web/src/AiPanel.tsx	the steps on <code>jobs.output</code> , the

			last 50, written in order and awaited before the row is finished; the model's argument stripped of <code>\p{C}</code> and cut to 80; GET <code>/documents/:id/run</code> for the newest run; the panel polls it and a reload picks it up
6	<code>packages/db/migrations/0020_rate_limits.sql</code>	<code>packages/db/src/limit.ts</code>	<code>rate_limits (key, win, hits) ;</code> <code>take()</code> is one upsert with <code>greatest(win)</code> ; the hit inside <code>createRun</code> 's transaction, before the insert, rolled back with a refused run; <code>Retry-After</code> from the database's clock; the usage view in one REPEATABLE READ snapshot
7	<code>apps/api/src/app.ts</code>	<code>apps/api/src/client-address.ts</code>	every route but the probes and the webhook: <code>user: <id></code> , else <code>address: <client></code> ; <code>mint: <user></code> before the document lookup; sign-in per email under a SHA-256; the client address as the rightmost <code>X-Forwarded-For</code> entry a trusted proxy vouched for

Four ideas carry the epic, and they are what an interviewer will press on. **Exactly-once is a property of the whole, never of a delivery** (sections 1 and 3): messages arrive at least once or at most once, never exactly once, so "once" is manufactured by an idempotent receiver: a key claimed by a primary key, an op id the room has already answered, a node the retry finds already there. The industry word is *effectively once*. **A heartbeat is not a lease** (section 2): lesson 7's lease said "I may write until this time"; a heartbeat only says "I was alive a second ago", and the decision to give the job away is made by whoever reads it, so a slow worker that was given up on must be *fenced* by something the write itself checks, the attempt number. **Postgres is the truth and Redis is a cache** (sections 4 and 6): a queue message is a hint that a row exists, a lease is a hint that a room is owned, and a counter that must be shared and must not fail open lives beside the rows it protects. **Who is asking is a trust decision** (section 7): an address in a header is written by whoever sent the header, and only the hop you trust can vouch for the one before it.

THIS REPO	JAVA WORLD	PYTHON WORLD
Idempotency-Key claimed by a primary key in the job's transaction; a replay answers from the row; 422 on a different request	Stripe's Idempotency-Key : 24 h, a replay returns the saved response, different parameters are an <code>idempotency_error</code> ; Spring has nothing built in (a table and a filter, as here)	Stripe's Python SDK sends one per request; Django has nothing built in; Celery's <code>task_id</code> dedupes the message, not the effect
a 1 s heartbeat on the row, a 15 s stale sweep, <code>attempts</code> as a fence	Spring Batch's <code>JobRepository</code> marks a step <code>STARTED</code> and a restart must be told to <code>allowStartIfComplete</code> ; Quartz's misfire handling; SQS's visibility timeout is the same idea from the broker's side	Celery <code>acks_late</code> + <code>visibility_timeout</code> (a slow task is redelivered and runs twice); Sidekiq Pro's <code>super_fetch</code> ; both re-run, neither fences
ids that are a function of (job, step, op), so a replay is deduplicated by the room	Kafka's idempotent producer: a sequence number per (producer, partition), the broker drops what it has seen	the same idea; Celery's tasks are asked to be idempotent and given no help
a fixed window in Postgres, one upsert, <code>Retry-After</code> to the second	Bucket4j and resilience4j's <code>RateLimiter</code> are in-process (per instance); a shared count needs Redis or JDBC	django-ratelimit: cache-backed fixed windows, failing open when the cache is away by default
<code>X-Forwarded-For</code> read from the right, only past a trusted proxy	Spring's <code>ForwardedHeaderFilter</code> believes the header from anyone unless you configure trusted proxies; Tomcat's <code>RemoteIpValve</code> has <code>internalProxies</code>	Django's <code>REMOTE_ADDR</code> is the peer; django-ipware guesses from the left unless told otherwise; gunicorn's <code>forwarded_allow_ips</code>

1 · Idempotency keys: the claim is the row, and the answer is the job as it now is

A browser presses "Ask the AI", the request leaves, and the answer never comes back: a proxy timed out, the tab lost its network, the api returned a 502 while restarting. The person presses again. Without a key, two runs exist (or one 409 and a confused person); with one, the retry is *the same request* and gets the same answer. Stripe's version of this is the model: the client mints a key per operation, the server stores what it did under the key for 24 hours, and a replay returns the stored result. The design question is where the key lives and how two requests carrying it *at the same moment* are told apart, because a retry is often a duplicate on the wire rather than a person's second press, and both arrive within milliseconds.

FROM `packages/db/migrations/0018_idempotency_keys.sql`

```
-- E9.1 (F27): "start AI run" and "ship" may be retried. The client sends an Idempotency-Key with the press; a retry
-- with the same key is answered with the job the first one made (or joined), never a second job. The PRIMARY KEY is
-- the check, not a look-then-insert: of two requests at the same moment, the second waits on the first's row and
-- then reads it. Scoped to (org, user): a key can only ever name a job of the caller's own org, made by the caller.
-- `request` is what was asked (queue, document, input): the same key with a different request is refused (422).
-- A key lives 24 hours (as Stripe's do): each claim first forgets that user's older keys, so a key may be used again
-- after that. It also goes with its job, its user and its org.
create table idempotency_keys (
  org_id      uuid not null references orgs (id) on delete cascade,
  user_id     uuid not null references users (id) on delete cascade,
  key         text not null check (key ~ '^[!~]{1,255}$'),
  request     jsonb not null,
  -- Null only inside the claiming transaction (the key is claimed BEFORE the job is made); never seen committed.
  job_id      uuid references jobs (id) on delete cascade,
  created_at  timestamptz not null default now(),
  primary key (org_id, user_id, key)
);
-- Back the cascades from jobs and users.
create index idempotency_keys_job on idempotency_keys (job_id);
create index idempotency_keys_user on idempotency_keys (user_id);
```

Read the primary key first: `(org_id, user_id, key)`. The key belongs to a user in an org, so a key from one tenant can never resolve to another tenant's job, and the same string from two users is two keys (the test "a key belongs to its user in its org" makes bob use ann's key and asks for his own job, then makes him use it on *her* document and asks 404). The `request` column stores what was asked: the queue, the document, the input. That is what lets the same key with a different instruction be refused as `422 idempotency_key_reused` rather than silently answered with the wrong job, which is what Stripe does too (an `idempotency_error`

when the parameters differ). `job_id` is nullable, and the comment says why: the key is claimed *before* the job exists, inside the transaction that will make it, and nobody outside that transaction ever sees the null.

FROM `packages/contracts/src/index.ts`

```
/**
 * F27: the `Idempotency-Key` header of POST /documents/:id/runs and /ship. Optional; a retry with the same key gets
 * the same job. 1 to 255 printable ASCII characters (a UUID per press is what the canvas sends), as Stripe allows.
 */
export const IdempotencyKey = z.string().regex(/^[\x21-\x7e]{1,255}$/, "1 to 255 printable ASCII characters");
```

FROM `apps/api/src/app.ts`

```
/** F27: the request's `Idempotency-Key`, if it sent one. A malformed one is refused, never ignored: the retry it was
meant to make safe would not be. */
function idempotencyKey(c: Context): string | undefined {
  const key = c.req.header("idempotency-key");
  if (key === undefined) return undefined;
  if (IdempotencyKey.safeParse(key).success) return key;
  throw new HTTPException(400, { res: fail(c, 400, "invalid_body", [{ field: "idempotency-key", message: "1 to 255
printable ASCII characters" }]) });
}
```

A malformed key is a 400, never ignored. The tempting shape is "if the key does not parse, proceed without one", and it is wrong for the same reason a malformed cookie is not "nobody": the retry the key was meant to make safe would then make a second job, and the client would believe it was protected. Refusing is the honest answer.

FROM `packages/db/src/index.ts`

```
/**
 * F27: `make` runs in a transaction that first claims (this user, `key`) in this org. A second request with the key
 * waits on that row, then is answered with the job the first one made: `{ replay }` (its id), never a second job.
 * "key_reused": the key was claimed for a different request. When `make` made no job (busy, no such document) the
 * claim is rolled back, and the retry that follows is free to use the key. Keys live 24 hours (migration 0018).
 */
const withKey = async <T>(userId: string, key: string, request: object, make: (via: PoolClient) => Promise<T>,
jobOf: (made: T) => string | undefined): Promise<{ made: T } | { replay: string } | "key_reused"> => {
  const params = [orgId, userId, IdempotencyKey.parse(key), JSON.stringify(request)];
  const client = await pool.connect();
  let broken: Error | undefined;
  try {
    await client.query("begin");
    await client.query("delete from idempotency_keys where org_id = $1 and user_id = $2 and created_at < now() -
interval '24 hours'", params.slice(0, 2));
    if ((await client.query("insert into idempotency_keys (org_id, user_id, key, request) values ($1, $2, $3, $4) on
conflict do nothing", params)).rowCount !== 1) {
      // A statement of its own, so it sees the winner's row, committed while the insert waited on it (read
committed).
      const found = await one(z.object({ same: z.boolean(), job_id: z.string() }), "select request = $4::jsonb as
same, job_id from idempotency_keys where org_id = $1 and user_id = $2 and key = $3", params, client);
      await client.query("commit");
      if (!found) throw new Error("idempotency key held by nobody");
      return found.same ? { replay: found.job_id } : "key_reused";
    }
    const made = await make(client);
    const jobId = jobOf(made);
    if (jobId === undefined) {
      await client.query("rollback");
      return { made };
    }
    await client.query("update idempotency_keys set job_id = $5 where org_id = $1 and user_id = $2 and key = $3",
[...params, jobId]);
    await client.query("commit");
    return { made };
  } catch (err) {
    await client.query("rollback").catch(() => undefined);
    broken = err instanceof Error ? err : new Error(String(err));
    throw err;
  } finally {
    client.release(broken);
  }
}
```

```

    }
};

```

This is the section's centre, and the drill plants its bug. `withKey` opens a transaction, sweeps this user's keys older than 24 hours (a ponytail named in the migration: rows of users who never return stay until they do), and then *claims* the key with `insert ... on conflict do nothing`. Under READ COMMITTED, a second transaction inserting the same primary key does not fail at once: it **blocks on the first transaction's row until it commits or rolls back**, then sees either a conflict (the winner committed) or a free key (the winner rolled back). That blocking is the whole mechanism. Twenty requests arrive together; one inserts and goes on to make the job; nineteen wait inside their `insert`; when the winner commits, each of the nineteen gets `rowCount 0`, reads the winner's row in *a statement of its own* (the comment says why: a statement started before the winner committed would not see it), and answers with the winner's job id. No lock was named, no retry loop written; the primary key did it.

Then the two branches that make the key *safe* as well as unique. If `make` made no job (the document was busy, or gone), the transaction **rolls back**, and the claim with it: the key was never used, and the retry that follows the 409 may use it to start the next run. If it did, the job id is recorded on the key and both commit together, so a crash between "job made" and "key recorded" cannot leave a job without its key (and a replay would then make a second one). The `client.release(broken)` at the end is pg's way of throwing a connection away rather than returning a possibly poisoned one to the pool.

FROM `packages/db/src/index.ts`

```

createRun: async ({ documentId, instruction, createdBy, idempotencyKey, limit }) => {
  const input = CreateRunBody.parse({ instruction }); // the same contract the reader uses, BEFORE the write
  if (!orgExists || !isId(documentId) || (createdBy !== undefined && !isId(createdBy))) return undefined;
  // F31: the hit first, then the run, in one transaction that commits only if the run was made. The hit holds the
  // org's counter row until then, so an org's run starts take turns (for the length of one insert).
  const make = async (via: PoolClient): Promise<Run | "busy" | { retryAfterSeconds: number } | undefined> => {
    const allowed = limit === undefined ? ({ ok: true } as const) : await take(`ai_run:${orgId}`, limit, via);
    return allowed.ok ? insertRun(documentId, input, createdBy, via) : { retryAfterSeconds:
allowed.retryAfterSeconds };
  };
  const made = (run: Awaited<ReturnType<typeof make>>): string | undefined => (typeof run === "object" && "id" in
run ? run.id : undefined);
  if (idempotencyKey === undefined) return inTx(make, (run) => made(run) !== undefined);
  if (createdBy === undefined) throw new Error("an idempotency key needs the user it belongs to");
  const keyed = await withKey(createdBy, idempotencyKey, { queue: "ai", documentId, input }, make, made);
  if (keyed === "key_reused") return keyed;
  return "made" in keyed ? keyed.made : one(RunRow, "select * from jobs where org_id = $1 and id = $2 and queue =
'ai'", [orgId, keyed.replay]);
},

```

FROM `apps/api/src/app.ts`

```

// F27: a retry with the same Idempotency-Key answers 201 with the run the first request made, read from its row
// (enqueued again: BullMQ drops an add for a job id it holds, and a claimed job cannot be claimed twice).
// F31: each run made is one hit on the org's limit, counted with the insert: over it, 429 with Retry-After, and the
// key is not used up (the refused run was rolled back with its claim). A replay is answered without a hit. The count
// lives in Postgres, like the run: if the database is away, no run can be made anyway, so the limit cannot fail open.
document.post("/runs", need("editor"), async (c) => {
  const { doc } = c.var;
  const key = idempotencyKey(c);
  const { instruction } = await body(c, CreateRunBody);
  const run = await db.forOrg(doc.orgId).createRun({ documentId: doc.id, instruction, createdBy: c.var.user.id,
idempotencyKey: key, limit: aiRunLimit });
  if (!run) return notFound(c); // the document was deleted in between
  if (run === "busy") return fail(c, 409, "run_in_progress");
  if (run === "key_reused") return fail(c, 422, "idempotency_key_reused");
  if ("retryAfterSeconds" in run) return limited(c, run.retryAfterSeconds);
  try {
    await enqueue({ queue: "ai", jobId: run.id, orgId: run.orgId });
  } catch (err) {
    // Still a 201: the run exists and the worker's sweep will pick it up. A retry without the
    // Idempotency-Key would make a second run.
    process.stderr.write(`${JSON.stringify({ level: "warn", path: c.req.path, message: `enqueue failed, left to the
sweep: ${describeError(err)}\n` })}\n`);
  }
}

```

```
    return c.json(run, 201);
  });
```

"As its row now is" is the sentence to remember. A replay is answered by `select * from jobs where id = $replay`, never from the queue's `add()` and never from a cached copy of the first response: the first test claims the run after making it and asserts the replay says `running`. Stripe stores the original response bytes; this repo stores the job's id and reads the job, because the job is the truth and the person retrying wants to know where it is now. The ship route does the same with 200 rather than 201, because a ship press may have *joined* a waiting ship rather than made one, and the key keeps naming the ship it pressed even after newer ships are queued behind it.

FROM `packages/db/src/index.ts`

```
startShip: async ({ documentId, createdBy, idempotencyKey }) => {
  if (!orgExists || !isId(documentId) || (createdBy !== undefined && !isId(createdBy))) return undefined;
  if (idempotencyKey === undefined) return pressShip(documentId, createdBy, pool);
  if (createdBy === undefined) throw new Error("an idempotency key needs the user it belongs to");
  const keyed = await withKey(createdBy, idempotencyKey, { queue: "ship", documentId, input: {} }, (via) =>
    pressShip(documentId, createdBy, via), (started) => started?.ship.id);
  if (keyed === "key_reused") return keyed;
  if ("made" in keyed) return keyed.made;
  const ship = await one(ShipRow, "select * from jobs where org_id = $1 and id = $2 and queue = 'ship'", [orgId,
    keyed.replay]);
  return ship && { ship, created: undefined };
},
```

FROM `packages/queue/src/index.ts`

```
// jobId = our job's id: a second add while the first is waiting is dropped by BullMQ. Finished
// messages are removed at once; Postgres keeps the history, and a re-delivery is refused there.
enqueue: (ref) => withinDeadline((queue) => queue.add(ref.queue, ref, { jobId: ref.jobId, removeOnComplete: true,
  removeOnFail: true }), ref.queue),
```

The replay *enqueues again* (a ponytail the commit names) and it is harmless twice over: BullMQ drops an `add` for a job id it already holds, and the worker's `claim()` is `queued` → `running` in one statement, so a job runs at most once per claim whatever the queue says. That is the second half of "effectively once": the delivery may repeat; the state transition cannot.

FROM `apps/web/src/api.ts`

```
/**
 * F27: a POST that makes a job. One press, one Idempotency-Key: when the answer is lost or is a 5xx, the retry sends
 * the same key and the api answers with the job the first attempt made, never a second one. A refusal is not retried,
 * and neither is a rate limit (F31): the same request a moment later is refused again.
 */
async function postJob<S extends z.ZodType>(path: string, schema: S, body?: unknown): Promise<z.infer<S>> {
  const key = { "idempotency-key": crypto.randomUUID() };
  try {
    return await post(path, schema, body, key);
  } catch (problem) {
    if (problem instanceof Refused || problem instanceof Limited) throw problem;
    return post(path, schema, body, key);
  }
}
```

The client's half is short on purpose. One press mints one `crypto.randomUUID()`; a lost answer or a 5xx is retried once with the same key; a 4xx is not retried (the same request a moment later is refused again), and neither is a 429 (section 6). A retry loop with backoff is the upgrade; one retry is what makes the common case, a dropped connection, invisible to the person.

FROM `apps/api/src/idempotency.int.test.ts`

```
test("twenty simultaneous starts with one key make ONE run, and all twenty are answered with it, from its row", async ()
=> {
  const doc = await document("ann@example.com");
  const key = crypto.randomUUID();
  queueDown = true; // nothing the queue says (or fails to say) reaches the answer
  const answers = await Promise.all(Array.from({ length: 20 }, () => run("ann@example.com", doc, key)));
  queueDown = false;
```

```

expect(answers.map((a) => a.status)).toEqual(Array<number>(20).fill(201));
const runs = await Promise.all(answers.map(async (a) => Run.parse(await a.json())));
const rows = await jobs(doc, "ai");
expect(rows).toEqual([{} id: rows[0]?.id, status: "queued" {}]);
expect(new Set(runs.map((r) => r.id))).toEqual(new Set([rows[0]?.id]));

// Later, the same key answers the run as its row NOW is, not as it was when it was made.
await ctx.db.jobStore().claim({ queue: "ai", jobId: runs[0]?.id ?? "", orgId: doc.orgId });
const again = await run("ann@example.com", doc, key);
expect(again.status).toBe(201);
expect(Run.parse(await again.json())).toMatchObject({ id: runs[0]?.id, status: "running" });
expect(await jobs(doc, "ai")).toHaveLength(1);
});

```

FROM `apps/api/src/idempotency.int.test.ts`

```

test("a key belongs to its user in its org: another tenant's same key is theirs, and never reads this job", async () => {
  const mine = await document("ann@example.com");
  const theirs = await document("bob@example.com");
  const sameUserOtherOrg = await document("ann@example.com");
  const key = "shared-key-1";
  const a = Run.parse(await (await run("ann@example.com", mine, key)).json());
  const b = await run("bob@example.com", theirs, key);
  expect(b.status).toBe(201);
  expect(Run.parse(await b.json()).id).not.toBe(a.id);
  const c = await run("ann@example.com", sameUserOtherOrg, key);
  expect(c.status).toBe(201);
  expect(Run.parse(await c.json()).id).not.toBe(a.id);
  // bob cannot reach ann's document with ann's key either: to him it does not exist.
  expect((await run("bob@example.com", mine, key)).status).toBe(404);
});

```

FROM `apps/api/src/idempotency.int.test.ts`

```

test("a start refused as busy does not use up its key: once the run ends, the same key starts the next one", async () => {
  const doc = await document("ann@example.com");
  const first = Run.parse(await (await run("ann@example.com", doc, crypto.randomUUID())).json());
  const key = crypto.randomUUID();
  const busy = await run("ann@example.com", doc, key, "next");
  expect(busy.status).toBe(409);
  expect(await error(busy)).toBe("run_in_progress");
  await finish(doc, first.id);
  const next = await run("ann@example.com", doc, key, "next");
  expect(next.status).toBe(201);
  expect(Run.parse(await next.json()).id).not.toBe(first.id);
});

```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>insert ... on conflict do nothing</code> and <code>rowCount !== 1</code>	<code>INSERT ... ON CONFLICT</code> through JDBC; JPA's <code>persist</code> throws on a duplicate after the flush, which is too late to branch on cleanly	<code>INSERT ... ON CONFLICT</code> with psycopg; the ORM's <code>get_or_create</code> is a look-then-insert with a retry on <code>IntegrityError</code>
a second transaction blocking on an uncommitted row with the same primary key	the same Postgres behaviour; with MySQL InnoDB too, with gap locks	the same
<code>client.release(broken)</code> : a poisoned connection is destroyed, not returned	HikariCP evicts on <code>SQLException</code> by its own rules	psycopg_pool's <code>putconn</code> discards a connection in a failed state
<code>{ made: T } { replay: string } </code> <code>"key_reused"</code> : three outcomes as one union	a sealed interface with three records, or an <code>Either</code>	a <code>Union</code> of dataclasses and a <code>Literal</code>
<code>Promise.all(Array.from({ length: 20</code> <code>}, () => run(...)))</code> : twenty requests in flight at once	an <code>ExecutorService</code> and a <code>CountDownLatch</code>	<code>asyncio.gather</code>
<code>crypto.randomUUID()</code> in the browser	<code>UUID.randomUUID()</code>	<code>uuid.uuid4()</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The key as columns on the jobs row was rejected.

A `idempotency_key` column with a partial unique index is the obvious shape and it fails twice. A ship press that *joins* the waiting ship makes no row and so has nowhere to keep its key, yet its replay must still answer that ship. And the key could not be claimed before the job exists, which is exactly what serialises two requests at the same moment without touching the busy-run and waiting-ship indexes: with the key on the row, the second request would race the first through those indexes instead and one of them would see a 409 that no person pressed.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A look-then-insert was rejected, and drill 2 plants it.

`select ... where key = $1`, then `insert` if nothing was found, is what every ORM's `get_or_create` does, and under twenty concurrent requests every one of them finds nothing and inserts. The unique constraint then fails nineteen of them, and the code has to catch the error and read again, which is the same statement the design starts with, written the long way round. `on conflict do nothing` makes the claim and the check one statement, and the blocking on the uncommitted row is what a look could never give.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A refused run rolls its claim back rather than recording the refusal.

Stripe records every outcome under the key, including errors, so a retry of a failed request gets the same failure. That is right for a payment (the client must not be charged on a retry of a declined card) and wrong for a busy document: the person who got a 409 waits for the run to end and presses again with a retry of the same request, and a recorded refusal would answer 409 for 24 hours. So a `make` that made nothing rolls the whole transaction back, and the key is as if never sent.

The principle: a delivery is never exactly once, so make the *effect* idempotent and let the delivery repeat. The key is claimed by the database's own uniqueness, in the same transaction as the work, before the work exists, so that two claims at one moment are serialised by a row lock nobody had to name. The replay reads the truth, not a memory of the first answer. And a claim that produced nothing is undone, so the key protects the retry it was sent for and nothing else.

2 · Heartbeats, the stale sweep, and the attempt as a fence

`kill -9` says nothing. A worker that is killed, or whose host is lost, or that the OOM killer takes, leaves its job's row saying `running`, and E3 built one unfinished run per document on that row: the document's next run is refused for ever, a sandbox job left running blocks the preview and keeps its container from the reaper (the E4.2b finding), a ship never ends. BullMQ has a stalled-job checker for this, and it was rejected in one sentence: Redis is not the truth, and a message can be dropped. So the job's own row says whether its worker is alive, and Postgres decides what to do about a silent one.

FROM packages/db/migrations/0019_job_heartbeat.sql

```
-- E9.2a (F28): a worker killed mid-job (kill -9, OOM, a lost host) says nothing, and its row stays `running`:
-- for an AI run that blocks the document's next run for ever (one unfinished run per document), for a sandbox
-- job it blocks the preview and keeps the container from being reaped. So a running job proves it is alive:
-- its worker writes heartbeat_at every second, and a sweep puts a job whose heartbeat went stale back in the
-- queue. `attempts` counts claims: it bounds the retries (a job that kills its worker every time must end),
-- and it FENCES a worker that was only slow: its heartbeat and its finish name the attempt they claimed,
-- and a newer claim makes both write nothing.
alter table jobs add column heartbeat_at timestampz;
alter table jobs add column attempts integer not null default 0 check (attempts >= 0);
-- The sweep asks "which running jobs went silent?" every few seconds; only running rows are in this index.
create index jobs_running on jobs (heartbeat_at) where status = 'running';
```

FROM packages/db/src/index.ts

```
/** `attempt`: the claim a worker holds (claim() returns it). Given, a write lands only while that claim is the job's
latest. */
type JobKey = { queue: (typeof QUEUES)[number]; jobId: string; orgId: string; attempt?: number | undefined };
/** A job as the worker sees it. `input` is whatever the creating route validated and stored. */
export type Job = { id: string; orgId: string; documentId: string; queue: JobKey["queue"]; input: Record<string, unknown>;
/** Undefined once that user has been deleted. */ createdBy: string | undefined };
type JobStore = {
  /**
   * queued -> running, atomically, as the job's next `attempt` (1 for the first). Undefined when there is nothing to
   * claim: unknown, already claimed, finished, or a job of ANOTHER queue.
   */
  claim(key: JobKey): Promise<Job & { attempt: number }> | undefined;
  /** running -> a terminal status. `reason` is what the user will read: anything that is not a plain name is stored as
  `internal`. */
  finish(key: JobKey, status: "succeeded" | "failed" | "cancelled", reason?: string): Promise<void>;
  /** The oldest jobs still waiting, across ALL orgs: what the worker offers to the queue again. */
  queued(limit: number): Promise<JobKey[]>;
  /**
   * "Still alive", from the worker running this attempt, once a second (F28), and the answer to "has someone asked
   * you to stop?" (F10). `lost`: the job is not this attempt's any more (finished, or given to another worker
   * after this one went silent): stop, and write nothing.
   */
  heartbeat(key: JobKey & { attempt: number }): Promise<"running" | "cancel" | "lost">;
  /**
   * Running jobs, across ALL orgs, whose heartbeat is older than `staleMs`: their worker died. Each goes back to
   * `queued` for another attempt, or, after `maxAttempts` claims, fails as `worker_lost` (one whose cancel was asked
   * for ends `cancelled`). `lost` counts both kinds of ending. A running ship with a
   * ship already waiting for its document fails too (the waiting one ships everything, and two may not wait).
   */
  requeueStale(staleMs: number, maxAttempts: number): Promise<{ requeued: number; lost: number }>;
}
```

Two columns. `heartbeat_at` is the last moment a worker said "still here", once a second. `attempts` counts claims and does two jobs at once: it **bounds** the retries (a job that kills its worker every time must still end, and after three claims it fails as `worker_lost`), and it **fences**. The fence is the part most retry systems miss. A worker that was only slow, a 20-second GC pause, an event loop blocked by a synchronous call, a network partition from Postgres, stops beating for longer than `staleMs` and is given up on; then it wakes,

finishes its work, and writes `succeeded` over a row that another worker is now running as attempt 2. Lesson 7 met the same shape with the lease token and answered it by making every write name the token; here every write names the attempt.

FROM `packages/db/src/index.ts`

```
// `queue = $3`: the message says which queue it came from, the ROW says which queue the job is
// on, and they must agree BEFORE anything is written. Without it, a message on the ai queue that
// names a git job would mark it running and then fail to parse it: running for ever.
claim: async ({ queue, jobId, orgId }) =>
  isId(jobId) && isId(orgId)
  ? one(JobRow, "update jobs set status = 'running', started_at = now(), heartbeat_at = now(), attempts = attempts
+ 1 where org_id = $1 and id = $2 and queue = $3 and status = 'queued' returning *", [orgId, jobId, queue])
  : undefined,
  async finish({ jobId, orgId, attempt }, status, reason) {
    if (!isId(jobId) || !isId(orgId)) return;
    // `status = 'running': a finished job stays finished, whoever reports late. `attempts`: a worker that went
    // silent, was given up on and woke up later does not end the attempt another worker is running now.
    await pool.query("update jobs set status = $3, error = $4, finished_at = now() where org_id = $1 and id = $2 and
status = 'running' and ($5::int is null or attempts = $5)", [
      orgId, jobId, status, status === "failed" ? (FailureReason.safeParse(reason).success ? reason : "internal") :
      null, attempt ?? null,
    ]);
  },
  async heartbeat({ jobId, orgId, attempt }) {
    if (!isId(jobId) || !isId(orgId)) return "lost";
    const beat = await one(
      z.object({ cancel: z.boolean() }),
      "update jobs set heartbeat_at = now() where org_id = $1 and id = $2 and status = 'running' and attempts = $3
returning cancel_requested_at is not null as cancel",
      [orgId, jobId, attempt],
    );
    return beat === undefined ? "lost" : beat.cancel ? "cancel" : "running";
  },
}
```

`claim` is the only way in: `queued` → `running`, `attempts + 1`, the first beat, in one statement that returns the row; a second message for the same job finds nothing to claim. `finish` and `heartbeat` both carry `and attempts = $attempt`: a beat from a dead attempt returns no row, which the store reads as `lost`, and a finish from one updates nothing. The beat also answers the cancel question (F10) that E3's poll used to ask, so the per-job timer does one round trip a second rather than two.

FROM `packages/db/src/index.ts`

```
async requeueStale(staleMs, maxAttempts) {
  // ONE statement, `skip locked`: every worker process sweeps, and two sweeps never both take a row.
  // coalesce(heartbeat_at, started_at): a row left running by a worker from before heartbeats (0019) is stale too.
  // `distinct on`: of the ships of one document that went silent together, only the newest may wait again
  // (0011: one waiting ship per document); it reads the document afresh, so it ships what they would have.
  // A job someone asked to stop is not run again: it ends as `cancelled` (F10), as its worker would have ended it.
  // A sandbox's address dies with its worker (the next attempt reports one); a ship's commit stays, as the git
  // peer asks it "did Ship push this?" (E5.5), and the push may still be on its way. An AI run's steps are the
  // dead attempt's (F30): the next attempt starts its list afresh, so a reload never shows steps nobody is taking.
  const counted = await one(
    z.object({ requeued: z.number().int(), lost: z.number().int() }),
    `with stale as (
      select id, queue, document_id, attempts, created_at, cancel_requested_at is not null as cancel from jobs
      where status = 'running' and coalesce(heartbeat_at, started_at) < now() - make_interval(secs => $1::float8 /
1000)

      for update skip locked),
      again as (
        select distinct on (case when queue = 'ship' then document_id else id end) id from stale s
        where not cancel and attempts < $2 and not (queue = 'ship' and exists (select 1 from jobs w where w.queue =
'ship' and w.status = 'queued' and w.document_id = s.document_id))
        order by case when queue = 'ship' then document_id else id end, created_at desc),
      requeued as (
        update jobs set status = 'queued', started_at = null, heartbeat_at = null, output = case when queue in
('sandbox', 'ai') then null else output end
        where id in (select id from again) returning 1),
      lost as (
        update jobs set status = case when stale.cancel then 'cancelled' else 'failed' end, error = case when
stale.cancel then null else 'worker_lost' end, finished_at = now()
        from stale where jobs.id = stale.id and stale.id not in (select id from again) returning 1)
      select (select count(*) from requeued)::int as requeued, (select count(*) from lost)::int as lost`,
```

```

    [staleMs, maxAttempts],
  );
  return counted ?? { queued: 0, lost: 0 };
},

```

The sweep is one statement, and every clause is a decision. `for update skip locked`: every worker process runs the sweep (the AI worker brings a dead sandbox worker's job back, and the other way round), and two sweeps at once must not both take a row, so each locks what it will touch and skips what another already holds; the test fires four sweeps over ten stale rows and counts ten queues.

`coalesce(heartbeat_at, started_at)`: a row left running before this migration has no beat and is judged by its start. `distinct on` for ships: of the ships of one document that went silent together only the newest may wait again, because migration 0011 allows one waiting ship per document, and the newest reads the document afresh so it ships what all of them would have. A job someone asked to cancel is not run again: it ends `cancelled`, as its worker would have ended it. And the `output` case: a sandbox's address dies with its worker, an AI run's steps are the dead attempt's (section 5), a ship's commit stays because the git peer will ask "did Ship push this?".

FROM `apps/worker/src/worker.ts`

```

/**
 * A running job whose last beat is older than this was left by a dead worker ('kill -9' says nothing): the sweep
 * gives it another attempt. Many beats long, so a busy event loop or a slow query is not taken for a death; and
 * if it is, the attempt fence makes the slow worker stop instead of finishing twice.
 */
staleMs?: number;
/** Claims a job may have in all. A job that kills its worker every time must still END: then it fails as 'worker_lost'.
 */
maxAttempts?: number;

```

FROM `apps/worker/src/worker.ts`

```

// Claiming is the ONLY way to start: queued -> running, in one statement. A duplicate or stale
// message finds nothing to claim and ends here, so a job runs at most once per claim.
const job = await jobs.claim(ref);
if (!job) return;
// Everything this attempt writes names it: once the job is someone else's, it writes nothing.
const mine = { ...ref, attempt: job.attempt };
// ponytail: a beat per running job per second (a handful at most). Redis pub/sub if a second ever matters.
// The signal only ASKS. "Ends within 3 s" is the handler's promise (ai.ts races its work against it):
// a handler for a new queue that ignores the signal holds its slot, and a polite shutdown, for ever.
// 'lost' aborts it too: this worker went silent for longer than staleMs and the job was given to another.
const cancel = new AbortController();
const watch = setInterval(() => {
  jobs.heartbeat(mine).then((state) => {
    if (state === "lost" && !cancel.signal.aborted) log("warn", "job taken over after a silence: stopping this
attempt", { jobId: ref.jobId, attempt: job.attempt });
    if (state !== "running") cancel.abort();
  }, () => undefined); // a failed beat is tried again in a second; staleMs is many of them
}, cancelPollMs);

```

The worker's half. `mine` is the key every write of this attempt carries. The interval beats, and the answer drives an `AbortController`: `cancel` aborts the handler (F10, as before); `lost` aborts it too, with a warning, which is how a slow worker learns it has been given up on and stops *before* writing anything over the next attempt. A failed beat is simply tried again next second: `staleMs` is fifteen of them, so one dropped query is not a death. That ratio is the design's one tunable, and the reasoning is the same as lesson 7's ttl: many beats long, so a busy loop or a slow query is not mistaken for a crash, and if it ever is, the fence makes the mistake safe rather than fatal.

FROM `apps/worker/src/worker.ts`

```

// BullMQ's own retries and stalled-job checks are not what brings a job back: Postgres is (the sweep below).
// A message whose worker died is either re-delivered by BullMQ (claim() refuses it while the row still says
// running, then takes it once the sweep has put the row back) or dropped, and the sweep offers the row again.
const workers = QUEUES.filter((name) => handlers[name] !== undefined).map((name) => {
  const worker = new Worker(name, (message) => run(message.data), { connection: connection(redisUrl), concurrency:
concurrency[name] ?? 4, ...scoped });
  worker.on("error", (err) => log("warn", describeError(err), { queue: name })); // without a listener, a Redis hiccup
is an uncaught exception

```

```
    return worker;
  });
}
```

FROM `apps/worker/src/worker.ts`

```
// Redis is not the truth (SPEC §2.9): it can be flushed, and the api can die between its INSERT
// and its enqueue. Whatever Postgres still calls 'queued' is offered again; jobId + claim() make
// a second offer harmless. ponytail: a poll; LISTEN/NOTIFY or an outbox if 'sweepMs' is ever too slow.
// First, F28: a job whose worker died (its beat went stale) is made 'queued' again, so this same pass offers it.
// Every worker process sweeps every queue: the AI worker brings back a dead sandbox worker's job, and the other way
round.
const producer = createProducer({ redisUrl, ...scoped });
let sweeping: Promise<void> | undefined;
const sweep = (): Promise<void> => (sweeping ??= sweepOnce().finally(() => (sweeping = undefined)));
async function sweepOnce(): Promise<void> {
  try {
    await producer.ping(); // with nothing queued the loop below never touches Redis, and "alive" would mean "Postgres
is up"
    const stale = await jobs.requeueStale(staleMs, maxAttempts);
    if (stale.requeued + stale.lost > 0) log("warn", "jobs left running by a dead worker", stale);
    // One refused offer must not hide the 99 behind it: it stays 'queued' and is offered again next time.
    for (const ref of await jobs.queued(100)) await producer.enqueue(ref).catch((err: unknown) => log("warn", `offer
failed: ${describeError(err)}`, { jobId: ref.jobId }));
    onAlive?();
  } catch (err) {
    log("warn", `sweep failed: ${describeError(err)}`);
  }
}
```

The sweep first calls `requeueStale` and then offers every `queued` row to Redis again, in the same pass, so a job that just went stale is picked up by whichever worker is alive within one sweep. `sweeping ??=` is the memoised-promise idiom from lesson 8's dummy hash: never two sweeps from one process at once. And a refused offer must not hide the 99 behind it, which is why the loop catches per row.

FROM `packages/db/src/job-heartbeat.int.test.ts`

```
test("the dead attempt's late writes land nowhere: its finish and its beat are fenced by the attempt", async () => {
  const jobs = t.db.jobStore();
  const key = await aJob();
  await jobs.claim(key);
  await silentFor(key.jobId, 60);
  await jobs.requeueStale(10_000, 3);
  await jobs.claim(key); // attempt 2, on another worker
  await jobs.finish({ ...key, attempt: 1 }, "failed", "worker_stopped"); // the first worker wakes up and reports
  expect(await jobs.heartbeat({ ...key, attempt: 1 })).toBe("lost");
  expect(await row(key.jobId)).toMatchObject({ status: "running", attempts: 2 });
  await jobs.finish({ ...key, attempt: 2 }, "succeeded");
  expect(await row(key.jobId)).toMatchObject({ status: "succeeded", error: null });
});
```

FROM `apps/worker/src/crash.int.test.ts`

```
test("a slow worker that was given up on stops its attempt when it wakes, and writes nothing over the next one", async ()
=> {
  const key = await aJob();
  let sawStop = false;
  await work((_job, cancelled) => new Promise( (_, reject) => {
    cancelled.addEventListener("abort", () => { sawStop = true; reject(new Error("stopped")); }, { once: true });
  }, 60_000);
  await producer.enqueue(key);
  await until(async () => (await row(key.jobId)).status === "running", "attempt 1 runs");
  // What the sweep does to a worker silent past staleMs AND another worker's claim after it, as one step: this
  // worker must not get the chance to claim the job again itself in between.
  await db.rawQuery("update jobs set attempts = attempts + 1, started_at = now(), heartbeat_at = now() where id = $1",
[key.jobId]);
  await until(() => Promise.resolve(sawStop), "attempt 1 was told to stop");
  await new Promise((r) => setTimeout(r, 300)); // its finish(cancelled) has had time to land, if it could
  expect(await row(key.jobId)).toMatchObject({ status: "running", attempts: 2 });
});
```

```
await db.db.jobStore().finish({ ...key, attempt: 2 }, "succeeded");
});
```

FROM packages/db/src/job-heartbeat.int.test.ts

```
test("stale ships of one document: the newest waits again with its commit kept; one with a ship already waiting fails",
  async () => {
    const jobs = t.db.jobStore();
    const older = await aJob("ship");
    await jobs.claim(older);
    const newer = await aJob("ship", older.documentId);
    await jobs.claim(newer);
    await jobs.report(newer, { commit: "c".repeat(40), pr: null });
    await silentFor(older.jobId, 60);
    await silentFor(newer.jobId, 60);
    expect(await jobs.requeueStale(10_000, 3)).toEqual({ requeued: 1, lost: 1 }); // two queued ships would break 0011
    expect(await row(newer.jobId)).toMatchObject({ status: "queued", output: { commit: "c".repeat(40), pr: null } }); // the
    // git peer still knows Ship's push
    expect(await row(older.jobId)).toMatchObject({ status: "failed", error: "worker_lost" });

    const third = await aJob("ship");
    await jobs.claim(third);
    await aJob("ship", third.documentId); // a press after it started: waiting
    await silentFor(third.jobId, 60);
    await jobs.requeueStale(10_000, 3);
    expect(await row(third.jobId)).toMatchObject({ status: "failed", error: "worker_lost" }); // the waiting ship ships
    // everything
  });
```

FROM packages/db/src/job-heartbeat.int.test.ts

```
test("two sweeps at once never both take a row", async () => {
  const jobs = t.db.jobStore();
  const keys = await Promise.all(Array.from({ length: 10 }, () => aJob()));
  for (const key of keys) {
    await jobs.claim(key);
    await silentFor(key.jobId, 60);
  }
  const results = await Promise.all(Array.from({ length: 4 }, () => jobs.requeueStale(10_000, 3)));
  expect(results.reduce((n, r) => n + r.requeued, 0)).toBe(10); // every row once: the rows of the tests above are all
  // queued or finished by now
  for (const key of keys) expect(await row(key.jobId)).toMatchObject({ status: "queued", attempts: 1 });
});
```

Heartbeats against leases, once more. Lesson 7's lease was a *promise with a deadline*: the holder knows for how long it may write, renews before then, and gives up on its own clock when it cannot. A heartbeat is a *report with no promise*: the worker says it is alive and has no idea whether anyone still believes it, and the reader decides. The lease is the right tool where the holder must act alone for a while (a room ordering ops without asking anyone); the heartbeat is enough where the work can be handed to someone else and the only harm of a wrong guess is a duplicate, which the fence turns into a no-op. Both need a token in the write: the lease token in the fence claim, the attempt in `finish`. SQS's visibility timeout is a lease the broker grants; Celery's `acks_late` with a `visibility_timeout` is a heartbeat-less version of this sweep that runs the slow task twice with nothing to stop the first.

FROM apps/worker/src/starvation.int.test.ts

```
// integration:no-starvation-under-saturation (E9.2a, SPEC F29). The ai queue is full: as many long AI runs as its
// concurrency, each editing its document all the time (one of them the document an engineer is about to push to),
// and more runs waiting behind them. A push must still reach the open canvas in under 5 s. Everything here is real
// but the model: the worker and its queue (Redis), the AI handler and its peer, the sync server, the git peer with
// its production poll (1 s) and git itself. They even share ONE process and event loop, which production does not
// give them (the git peer is its own service): if the push gets through here, it gets through there.
```

F29, no starvation. The git peer is its own process and its own loop, so a saturated AI queue cannot hold a push up; the integration test proves it in the hardest configuration, everything on one event loop, and asks a push to reach an open canvas in under five seconds while every AI slot is busy editing.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>new AbortController()</code> aborted by the beat's answer	a <code>volatile boolean</code> and <code>Thread.interrupt()</code> , or a <code>CompletableFuture.cancel</code>	<code>asyncio.Event</code> , or a <code>threading.Event</code>
<code>setInterval(...)</code> per running job, cleared in <code>finally</code>	a <code>ScheduledExecutorService</code> task per job	<code>asyncio.create_task</code> with a loop and <code>sleep</code>
sweeping ??= <code>sweepOnce().finally(...)</code> : one sweep at a time	a <code>ReentrantLock.tryLock()</code> around the sweep	an <code>asyncio.Lock</code> with <code>locked()</code> checked first
<code>for update skip locked</code>	the same SQL; JPA's <code>@Lock(PESSIMISTIC_WRITE)</code> cannot say <code>skip locked</code> portably	the same SQL; <code>select_for_update(skip_locked=True)</code> in Django
<code>make_interval(secs => \$1::float8 / 1000)</code> : milliseconds from the caller, an interval in SQL	the same	the same
<code>distinct on (case when ... then document_id else id end)</code> : one of a group, chosen by <code>order by</code>	a window function with <code>row_number()</code>	the same SQL
<code>(job.attempt)</code> : a number typed into every write's key	a version column and <code>@Version</code> : optimistic locking is the same fence	a version column checked in the <code>UPDATE ... WHERE</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

BullMQ's stalled-job retries were rejected.

BullMQ can notice a worker that stopped renewing its lock and re-deliver the message. It lives in Redis, which section 4 wipes on purpose, and it knows nothing about the row: a re-delivered message for a job the row still calls `running` would be refused by `claim()` anyway. Putting the heartbeat on the row makes the row self-sufficient, and makes the stalled check the same statement on every worker.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The fence exists because of a worker that was only slow.

The first version had the heartbeat and the sweep and no attempt number: a job silent past 15 s was requeued, and the dead worker was assumed dead. A test then held a worker's handler open, bumped the row to attempt 2 by hand as the sweep and another claim would, and released it: the slow worker's `finish("succeeded")` landed on the row attempt 2 was running, and the second worker's later finish then updated nothing because the status was no longer `running`. Naming the attempt in every write, and telling the slow worker `lost` on its next beat, made both writes land where they should: nowhere and on the row respectively.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The sweep's ship clause was found by 0011, not by design.

Requeueing every stale ship would have inserted two `queued` ships for one document when two had gone silent together, and the partial unique index from E5 (one waiting ship per document) would have failed the whole sweep statement, for every queue. The `distinct on` picks the newest, the older one fails as `worker_lost`, and a running ship whose document already has a ship waiting fails too, because the waiting one will ship everything. Each of those cases is one line of the heartbeat test.

The principle: a retry system has three parts and most have two. *Detection:* something says the worker is gone (a heartbeat read by a sweep). *Redelivery:* the work is offered again (the row back to `queued`, and a bound on how often). *Fencing:* the write path refuses the worker that was wrongly given up on (the attempt in every `update ... where`). Without the third, every false positive of the first is a duplicate, and the first has false positives by construction, because a timeout cannot tell slow from dead.

3 · Replaying a dead attempt without a duplicate: ids that are functions of the step

Attempt 2 of an AI run starts its conversation with the model over, because the run's transcript is not stored (SPEC §6). The model, given the same instruction and the same document, will usually do the same first steps again, and each of them is an `add_node` the dead attempt already sent to the room. Without care the document ends with every early node twice. The tempting fix, "let the retry read the document and skip what is there", asks the model to be idempotent, which no model is. The repo's answer is to make the *ids* idempotent instead: the n-th node id and the n-th op id of a job are functions of the job and the step, the same on every attempt, so a replayed `add_node` either finds its node already there or is refused by the room as a duplicate op the room already answered.

FROM `apps/worker/src/ai.ts`

```
/**
 * The ids a run mints, the same on every attempt of its job (F28): the n-th node is always the same id, and the n-th
 * op the same opId when it is the same op. A retry after a crash replays the dead attempt's steps first; a node it
 * finds already there is that step done, and an op the room journaled already gets its original answer (the
 * journal's key is (sender, opId)), so the document never holds anything twice. The op's content is in its id: a
 * model that does something ELSE at step n on the retry sends a new op, never one the room would take for the old.
 */
export function replayIds(jobId: string): { opId: (op: Op) => string; nodeId: () => string } {
  let ops = 0;
  let nodes = 0;
  const digest = (text: string): string => createHash("sha256").update(text).digest("hex");
  return {
    // Shaped as a version-8 (custom) UUID, which the contract's z.uuid() accepts.
    opId: (op) => {
      const h = digest(`${jobId}:op:${String(ops++)}:${JSON.stringify(op)}`);
      return `${h.slice(0, 8)}-${h.slice(8, 12)}-8${h.slice(13, 16)}-${((parseInt(h.slice(16, 17), 16) & 0x3) |
0x8).toString(16)}${h.slice(17, 20)}-${h.slice(20, 32)}`;
    },
    nodeId: () => `n_${digest(`${jobId}:node:${String(nodes++)}`).slice(0, 12)}`,
  };
}
```

Read the op id's input: `jobId:op:n:JSON.stringify(op)`. The *content* of the op is in its id, on purpose, and the commit's reasoning is worth keeping whole: with `(job, n)` alone, a model that does something *else* at step n on the retry would send a different op under an id the room already journaled, the room would answer with the old op's seq (E6.1's dedupe answers a known `(sender, opId)` from the journal), the replica would never match that answer to its op, and it would resend for ever. Putting the op's bytes into the hash makes "a different op at the same step" a different id: a new op, refused or accepted on its own merits. The id is shaped as a version-8 UUID because the contract says `z.uuid()` and the room's dedupe key is that string.

FROM `apps/worker/src/ai.ts`

```
const ids = replayIds(job.id);
const peer = connectPeer({
  manifest,
  mintOpId: ids.opId,
  // The worker holds the signing secret, so it mints its own session. The ROOM stamps every op
  // with this actor; nothing the agent sends can claim to be a person, or another run.
  session: async () => ({
    wsUrl: await roomUrl(sessions, job.documentId),
    token: signSessionToken({ userId, orgId: job.orgId, documentId: job.documentId, secret: sessions.secret,
ttlSeconds: 60, actor: { kind: "agent", runId: job.id } }},
  }),
});
```

FROM `packages/peer-client/src/replica.ts`

```
export function createReplica({ manifest, maxPending = 2000, window = 50, mintOpId = () => crypto.randomUUID() }: {
manifest: Manifest; maxPending?: number; /** How many unanswered ops may be on the wire at once. */ window?: number; /**
```

```
Injected, like the room's clock: the simulator needs every run of a seed to be identical, and an AI run retried after a crash its own ids again (F28). */ mintOpId?: (op: Op) => string }
```

`mintOpId` was already an injection point: the reconcile simulator (F8a) needs every run of a seed to be identical, so the replica never minted its own ids unconditionally. E9.2a gives it a second caller and passes the `op`, because the id now depends on it. That is a small lesson in seams: a dependency injected for a test is a dependency you can reuse for a feature, and one you hard-coded is a refactor.

FROM `apps/worker/src/tools.ts`

```
/**
 * One op, start to finish: the replica's verdict, then the ROOM's. Either refusal is a tool error (F11).
 * 'replayed': asked when the replica says the node already exists. True = this step's node, added by an
 * earlier attempt of the same run: the step is done, not refused.
 */
async function apply(op: Op, done: unknown, replayed = (): boolean => false): Promise<ToolResult> {
  const submitted = peer.submit(op);
  if (!submitted.ok && submitted.reason === "duplicate_node" && replayed()) return ok(done);
  if (!submitted.ok) return refused(submitted.reason, hints.get(submitted.reason));
  const outcome = await submitted.settled;
  return outcome.ok ? ok(done) : refused(outcome.reason, hints.get(outcome.reason));
}
```

FROM `apps/worker/src/tools.ts`

```
tool(
  "add_node",
  'Adds a component instance under a parent and returns its new nodeId. Components: ${components}.' ,
  { parentId: nodeId, component: z.string(), props: z.object({}).catchall(value).default({}).describe("prop name -> value; may be omitted"), index: z.number().int().optional().describe("position among the parent's children; omit to append") },
  ({ parentId, component, props, index }) => {
    const id = mintNodeId(); // the tool mints ids: a model would reuse "card1" across runs
    // Same component under the same parent: the dead attempt's node for this step (a model that changed its mind gets the refusal, and a new id next time).
    const replayed = (): boolean => { const there = nodeOf(peer.doc, id); return there?.component === component && there.parentId === parentId; };
    return apply({ type: "add_node", nodeId: id, parentId, component, props, index: index ?? nodeOf(peer.doc, parentId)?.children.length ?? 0 }, { nodeId: id }, replayed);
  },
),
```

The node side. `add_node` mints the same id at the same step; the replica, holding the document the room sent at welcome, sees that id already exists and refuses with `duplicate_node` before anything goes on the wire; and the tool asks `replayed()`: is the node that is there the same component under the same parent? If so, this step is *done*, the model is told the node id it would have been told, and nothing is sent. If not (the model changed its mind at this step), the refusal stands, the model hears `duplicate_node`, and its next attempt to add gets a fresh id from the next step. The unit test pins both.

FROM `apps/worker/src/replay.test.ts`

```
test("a retried attempt that replays add_node finds the dead attempt's nodes: done, and nothing is added twice", async () => {
  const job = randomUUID();
  const doc = { current: emptyDoc() };
  const dead = peerOver(doc);
  const died = buildTools(dead, manifest, replayIds(job).nodeId);
  const one = await tool(died, "add_node").run({ parentId: "root", component: "Stack" });
  const two = await tool(died, "add_node").run({ parentId: "root", component: "Button", props: { label: "Pay" } });
  expect([one.ok, two.ok]).toEqual([true, true]);

  const retry = peerOver(doc);
  const again = buildTools(retry, manifest, replayIds(job).nodeId);
  expect(await tool(again, "add_node").run({ parentId: "root", component: "Stack" })).toEqual(one); // the same answer, the same nodeId
  expect(await tool(again, "add_node").run({ parentId: "root", component: "Button", props: { label: "Pay" } })).toEqual(two);
  expect(retry.sent).toEqual([]); // nothing reached the room
  expect(Object.keys(doc.current.nodes)).toHaveLength(3); // root + the two
```

```
// The retry goes on past where the dead attempt stopped: a new step, a new node.
expect(await tool(again, "add_node").run({ parentId: "root", component: "Stack" })).ok().toBe(true);
expect(Object.keys(doc.current.nodes)).toHaveLength(4);
});
```

FROM `apps/worker/src/replay.test.ts`

```
test("the op's content is in its id: something else at the same step is a new op, never the old one's answer", () => {
  const job = randomUUID();
  expect(replayIds(job).opId(add("a"))).not.toBe(replayIds(job).opId(add("b")));
  // The step counts too: the same op again later is its own op (a set_prop back and forth is two edits).
  const ids = replayIds(job);
  expect(ids.opId(add("a"))).not.toBe(ids.opId(add("a")));
});
```

The other two queues. Ship needs nothing new: a branch that already holds the page gets no commit, and Gitea's 409 on an existing pull request finds the open one (E5.5's idempotence), so a retried ship makes no second PR. Sandbox needs one thing: the stale sweep clears the dead worker's address (the `output` case in section 2), so the document's preview comes back under the next attempt instead of pointing at a container nobody runs. The chaos check does all three rounds with a real `kill -9`.

FROM `scripts/chaos/kill-worker-resumes.ts`

```
// chaos:kill-worker-resumes (E9.2a, SPEC F28 and §4 "worker dies"). Against the REAL compose stack:
// ./init.sh (or: docker compose up -d), then node scripts/chaos/kill-worker-resumes.ts
// Three rounds, one per kind of job, each ending with a `kill -9` of the worker in the middle of it:
// ai:      a run of 20 steps on the stub worker (scripts/chaos/stub-worker.ts: the real worker and AI handler, a
//          scripted agent; the compose `worker` is stopped meanwhile so no model is ever called). Killed after a
//          few steps: the row stays `running` (nobody said anything), no edit arrives while it is dead, a second
//          stub worker takes it once its heartbeat is stale, and it SUCCEEDS as attempt 2 with the document holding
//          each of the 20 nodes exactly once and the journal every op once.
// sandbox: a document open with its preview up; worker-sandbox is killed and started again: the preview's job is
//          taken again (attempt 2) and the canvas's preview comes back, instead of the document being blocked for
//          ever (the E4.2b finding).
// ship:    worker-ship is killed as soon as a ship runs, and started again: the ship ends `succeeded` and Gitea has
//          exactly ONE open pull request for the document (Ship's own idempotence, E5.5).
// Waits poll the observable, never a fixed sleep; every service is started again in `finally`. Prints one JSON line;
// exit 0 = PASS. Cleanup: the org (cascade), the stub containers, the ship's branch in Gitea.
```

FROM `scripts/chaos/stub-worker.ts`

```
const scripted: RunAgent = async ({ instruction, tools, signal }) => {
  const count = Number(/^nodes=(\d{1,3})$/u.exec(instruction)?.[1] ?? "0");
  const add = tools.find((each) => each.name === "add_node");
  if (!add) throw new Error("no add_node tool");
  for (let step = 0; step < count && !signal.aborted; step++) {
    const result = await add.run({ parentId: "root", component: "Stack" });
    if (!result.ok) throw new Error(`step ${String(step)}: ${result.text}`); // a replayed step must be DONE, never
    refused
    await new Promise((r) => setTimeout(r, stepMs));
  }
  return { model: "stub", inputTokens: 0, outputTokens: 0, cacheReadTokens: 0, cacheWriteTokens: 0, costUsd: 0 };
};
```

FROM `scripts/chaos/kill-worker-resumes.ts`

```
docker("kill", "--signal", "SIGKILL", first);
const atKill = job(run);
const journaledAtKill = agentRows();
check(atKill.status === "running" && atKill.attempts === 1, `a killed worker leaves its job running as attempt 1 (got
${JSON.stringify(atKill)})`);
check(journaledAtKill < steps, `killed mid-run: ${String(journaledAtKill)} of ${String(steps)} steps done`);
await new Promise((resolve) => setTimeout(resolve, 2000));
check(agentRows() === journaledAtKill && job(run).status === "running", "nothing moves while no worker is alive");

await startStub();
await until(() => ![ "queued", "running" ].includes(job(run).status), "the retried job ends", STALE_MS + 90_000);
const done = job(run);
check(done.status === "succeeded" && done.attempts === 2, `the job completes as attempt 2 (got
```

```

${JSON.stringify(done)}}`);
// The second attempt started after the first one's last beat had gone stale, not before: a live job is never taken.
const retriedAfterMs = done.started - atKill.heartbeat;
check(retriedAfterMs >= STALE_MS, `retried ${String(retriedAfterMs)} ms after the last heartbeat, which is not stale yet`);

// No duplicate ops: every step's node once, every op once, the room's numbering whole.
const [ops = "0", opIds = "0", nodes = "0", seqs = "0", maxSeq = "0"] = psql(
  `select count(*), count(distinct op_id), count(distinct op->'nodeId'), count(distinct seq), coalesce(max(seq), 0)
from op_journal where document_id = '${doc}' and actor_kind = 'agent' and run_id = '${run}'`,
).split("|");
check(Number(ops) === steps && Number(opIds) === steps && Number(nodes) === steps, `the journal holds ${String(steps)}
add_node ops, each op and each node once (got ops=${ops} opIds=${opIds} nodes=${nodes})`);
check(Number(seqs) === Number(maxSeq) && Number(maxSeq) === steps, `seq 1..${String(steps)} with no gap (got ${seqs}
distinct, max ${maxSeq})`);

```

Read what the AI round measures. It kills the stub worker container after five journaled steps, asserts that the row still says `running`, `attempts = 1` and that *nothing moves* for two seconds while no worker is alive (a check that the job is not being run by something the script did not start), starts a second stub, and waits. Then: succeeded as attempt 2; the second attempt's `started_at` at least `staleMs` after the first's last `heartbeat_at`, both by Postgres's clock; the journal holds exactly 20 `add_node` ops with 20 distinct op ids and 20 distinct node ids for this run; seqs 1..20 with no gap; and a fresh peer's document holds exactly 20 Stacks. The stub is the real worker loop, the real AI handler and the real peer with a scripted agent in place of the model, running inside the worker image on the compose network, so the only thing not real is the model, and the model is the one part whose behaviour a retry cannot rely on anyway.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>createHash("sha256").update(text).digest("hex")</code> sliced into a UUID shape	<code>UUID.nameUUIDFromBytes</code> (version 3, MD5): the same idea, a name-based UUID	<code>uuid.uuid5(namespace, name)</code>
a closure over two counters (<code>ops</code> , <code>nodes</code>): state per run, no class	an object with two <code>int</code> fields	a closure or a small class
<code>replayed = (): boolean => ...</code> as a default parameter	an overload with a <code>BooleanSupplier</code>	a keyword argument with a lambda default
<code>mintOpId?: (op: Op) => string</code> : an optional injected function	a functional interface passed to the constructor	an optional callable parameter
<code>docker("kill", "--signal", "SIGKILL", name)</code> from a script	<code>ProcessBuilder</code>	<code>subprocess.run</code>
<code>psql(...).split(" ")</code> with defaults in the destructuring	JDBC in the harness	psycopg in the harness

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The op id as (job, step) alone was rejected: a resend for ever.

The first design hashed the job id and the step number. The refutation was a scenario, not a test: attempt 2's model adds a Button where attempt 1 added a Stack at step 3; the room finds the op id in its journal and answers with the Stack's seq; the replica has no pending op whose content that answer matches and keeps the Button pending; the ack timeout fires and it resends, and the room answers the same thing again. Adding the op's JSON to the hash costs nothing and makes the case impossible: a different op is a different id.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Skipping steps by reading the tree was rejected.

"Read the document first and do not add what is there" is an instruction to the model, and a model that follows instructions perfectly is not a model. Worse, it would have made the retry's behaviour depend on the prompt. The ids make the replay safe whatever the model does: the same step is absorbed, a different step is a new op.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A different plan on the retry is absorbed only where it repeats itself.

The ponytail is named in the commit: attempt 2 that adds different nodes adds them, and the run then shows what both attempts did. The ceiling is a document with some extra nodes after a crash; the upgrade is to resume the agent from its recorded tool calls, which is what E9.4's progress list is the first step towards.

The principle: when the producer of a retried action cannot be made deterministic, make its *identifiers* deterministic and let the consumer deduplicate. Kafka's idempotent producer does it with a sequence number per partition; here the id is a hash of (job, step, content), the replica dedupes nodes and the room dedupes ops. The content belongs in the id whenever "the same step, a different action" must be told apart from "the same action again".

4 · Redis is a cache: the wipe, and why no code changed

E9.2b is the epic's shortest bead and its most instructive: *no production code changes*. SPEC §4 asks that a Redis lost (flushed or restarted with no persistence) loses no job and no room, and the answer had been built piece by piece since epic 3. What was missing was the proof, and the proof is a chaos script with two rounds and a pure judge. Read the list of what already made it true, because each item is a place where an earlier epic chose Postgres over Redis for the truth.

Jobs. The worker's sweep (E3.1) re-offers every row Postgres still calls `queued`, every five seconds in compose; `jobId` as the BullMQ id plus `claim()` make a second offer harmless. A *running* job heartbeats Postgres, not Redis, so the wipe never notices it; BullMQ loses the job's lock and cannot record its end, which it logs, and Postgres records it, which is what counts. **Rooms.** After a wipe the owner's compare-and-renew finds no key and gives the room up at once (4409 to its peers), the next acquire takes the token floor from the journal's fence (E7.3), so the token keeps rising and no second owner can write, and the routing heartbeats come back within a tenth of a ttl. **Roles.** The E8.2 pub/sub subscriber re-reads every role (`"all"`) on reconnect, and a FLUSHALL keeps subscriptions alive anyway.

FROM `scripts/chaos/redis-wipe-rebuild.ts`

```
// chaos:redis-wipe-rebuild (E9.2b, SPEC F28 and §4 "Redis lost"). Against the REAL compose stack:
// ./init.sh (or: docker compose up -d), then node scripts/chaos/redis-wipe-rebuild.ts
// Two rounds, one per way of losing Redis: 'flush' (FLUSHALL: every key gone, every connection still open, so
// nobody is told) and 'restart' (the container restarts; it has no volume and no AOF on purpose). In each round:
// - AI runs on the stub worker (scripts/chaos/stub-worker.ts; the compose 'worker' is stopped so no model is called):
//   four long runs fill its four slots and are mid-way, three more wait in Redis's 'wait' list, and only there;
// - a room is open: two people editing, acknowledged edits, a burst on the wire, and a WITNESS (a peer that never
//   reconnects) in the owner's room;
// then Redis is wiped under all of it. Afterwards (scripts/chaos/rebuild.ts, scripts/chaos/no-loss.ts judge):
// - the waiting runs were offered again from Postgres (the worker's sweep) and every run succeeded, claimed ONCE,
//   each step journaled once: no duplicate job, no duplicate op;
// - the room's owner gave it up (the witness is closed), a node holds the lease again under a larger token (the floor
//   from the journal's fence, E7.3) and the fence names exactly that token: one owner. No acknowledged edit lost or
//   doubled, every peer on the same document.
// Waits poll the observable; the stack is put back in 'finally'. Prints one JSON line; exit 0 = PASS. The org is deleted.
```

FROM `scripts/chaos/redis-wipe-rebuild.ts`

```
type Wipe = "flush" | "restart";
async function wipe(how: Wipe): Promise<void> {
```

```

if (how === "flush") {
  const answer = redis("FLUSHALL");
  if (answer !== "OK") throw new Error(`FLUSHALL answered ${answer}`);
  return;
}
const before = runId();
compose("restart", "redis");
await until(() => { try { return redis("PING") === "PONG"; } catch { return false; } }, "Redis answers again", 60_000);
if (runId() === before) throw new Error("Redis did not restart: same run_id");
}

```

Two ways of losing Redis, because they are different failures. `FLUSHALL` empties every key while every connection stays open: nobody is told, subscriptions survive, and a client with a stale idea of what it holds finds out only when it asks. A *restart* drops every connection too, so every client reconnects and BullMQ's blocking pops return with errors. The script checks that the restart really happened (`run_id` changed) so a round cannot pass by doing nothing.

FROM `scripts/chaos/rebuild.ts`

```

// What chaos:redis-wipe-rebuild (E9.2b, SPEC §4 "Redis lost") judges, as pure checks: Redis is wiped while AI runs
// are running and others are waiting in it, and while a room is open. Postgres is the truth, so afterwards:
// - every run succeeded, claimed ONCE (a wipe must not start a job twice), each of its steps journaled once, the
//   document's seqs 1..n without a gap;
// - the room has one owner again: a lease with a token above the one before the wipe, and the journal's fence
//   (Postgres) naming that same token, so no second node holds a claim it could still write under.
// Vacuity guards (SPEC §4a): no job waiting in Redis at the wipe, or none running, proved nothing, and says so.

/** One run after the wipe: its row, and its document's journal counted (its own ops, and the room's seqs). */
export type RunAfter = { id: string; status: string; attempts: number; ops: number; opIds: number; nodes: number; seqs:
number; maxSeq: number };
export type Holder = { token: number; node: string };

export function runViolations({ steps, runs, waitingAtWipe, runningAtWipe }: { steps: ReadonlyMap<string, number>; runs:
readonly RunAfter[]; waitingAtWipe: number; runningAtWipe: number }): string[] {
  const violations: string[] = [];
  if (waitingAtWipe === 0) violations.push("vacuous: no job was waiting in Redis at the wipe, so no queue had to be
rebuilt");
  if (runningAtWipe === 0) violations.push("vacuous: no job was running at the wipe");
  for (const [id, want] of steps) {
    const run = runs.find((r) => r.id === id);
    if (!run) {
      violations.push(`run ${id} has no row`);
      continue;
    }
    if (run.status !== "succeeded") violations.push(`run ${id} ended ${run.status}, not succeeded`);
    if (run.attempts !== 1) violations.push(`run ${id} was claimed ${String(run.attempts)} times: the wipe started it
again`);
    if (run.ops !== want || run.opIds !== want || run.nodes !== want) violations.push(`run ${id} journaled
ops=${String(run.ops)} opIds=${String(run.opIds)} nodes=${String(run.nodes)}, not ${String(want)} of each`);
    if (run.seqs !== run.maxSeq) violations.push(`run ${id}'s document has ${String(run.seqs)} distinct seqs up to
${String(run.maxSeq)}: a gap or a repeat`);
  }
  return violations;
}

```

FROM `scripts/chaos/rebuild.ts`

```

export function leaseViolations({ before, after, fenceToken }: { before: Holder; after: Holder | undefined; fenceToken:
number }): string[] {
  if (!after) return ["nobody holds the room's lease after the wipe"];
  const violations: string[] = [];
  if (after.token <= before.token) violations.push(`the lease's token went from ${String(before.token)} to
${String(after.token)}: a flushed counter issued a token again`);
  if (fenceToken !== after.token) violations.push(`the journal's fence is at token ${String(fenceToken)} but the lease is
${String(after.token)}:${String(after.node)}: two owners`);
  return violations;
}

```

The judge is a pure function over what the script read from Postgres and Redis afterwards, and it has a unit test of its own (`rebuild.test.ts`, mutation-checked on the `attempts` and fence guards): every run `succeeded` with `attempts = 1`, because a

wipe must *not* start a job twice (that is section 2's job, and a wipe is not a dead worker); every step journaled once with distinct op ids and node ids; seqs with no gap; a lease back under a *larger* token; and `documents.fence_token` naming exactly that token, which is the one-owner property from lesson 7. The two vacuity guards are SPEC §4a's rule: a round in which nothing was waiting in Redis, or nothing running, proved nothing, and says so instead of passing.

FROM `apps/worker/src/crash.int.test.ts`

```
test("Redis wiped under a working worker: the running job ends once, the waiting ones are offered again from Postgres and run once", async () => {
  const held = await aJob();
  const waiting = [await aJob(), await aJob()];
  const calls: string[] = [];
  let release = (): void => undefined;
  // One at a time: 'held' runs, the other two wait in Redis, and only there.
  await work((job) => {
    calls.push(job.id);
    return job.id === held.jobId ? new Promise((resolve) => { release = () => { resolve(undefined); }; }) :
    Promise.resolve(undefined);
  }, 60_000, 1);
  const inspect = new Queue("ai", { connection: connection(TEST_REDIS_URL), prefix });
  try {
    await until(async () => (await row(held.jobId)).status === "running" && (await inspect.getWaitingCount()) === 2, "one job runs, two wait in Redis");
    // The wipe: every key of this file's prefix, as FLUSHALL would (the test Redis is shared, so not FLUSHALL itself).
    // One script, so it is atomic as FLUSHALL is: nothing lands between listing the keys and deleting them.
    const redis = await inspect.client;
    redis.defineCommand("wipe", { numberOfKeys: 0, lua: "local keys = redis.call('KEYS', ARGV[1]) for _, key in ipairs(keys) do redis.call('DEL', key) end return #keys" });
    expect(await redis.runCommand("wipe", [`${prefix}:*`])).toBeGreaterThan(0);
    release(); // BullMQ can no longer record this job's end; Postgres can, and that is the one that counts
    await until(async () => (await Promise.all([held, ...waiting].map(async (key) => (await row(key.jobId)).status))).every((s) => s === "succeeded"), "all three succeeded");
  } finally {
    await inspect.close();
  }
  expect(calls.toSorted()).toEqual([held, ...waiting].map((key) => key.jobId).toSorted()); // each ran once: no duplicate job
  for (const key of [held, ...waiting]) expect(await row(key.jobId)).toMatchObject({ status: "succeeded", attempts: 1 });
});
```

The integration version wipes only its own prefix (the test Redis is shared), with one Lua script so the wipe is atomic as `FLUSHALL` is: nothing can land between listing the keys and deleting them. It then releases the running handler and asserts every job succeeded, each handler was called once, `attempts = 1`.

QUESTION	THIS REPO	JAVA WORLD	PYTHON WORLD
What a wipe loses	the <code>wait</code> lists, the locks, the lease keys, the routing heartbeats, the pub/sub state (on restart)		
What brings jobs back	the sweep: <code>select ... where status = 'queued'</code> every 5 s, offered again; <code>claim()</code> refuses a duplicate	Celery with a Redis broker: the messages are gone and the tasks never run unless you kept your own table	Sidekiq: jobs in Redis are the truth, so a flush is data loss; Sidekiq Pro's reliable fetch survives a crash, not a flush
What brings rooms back	the owner's renew finds no key and closes with 4409; the next acquire's floor is the journal's fence	ZooKeeper and etcd persist their state (Raft log), so a restart is not a wipe; a Redisson lock on a flushed Redis is simply gone and the holder does not know	
What a running job feels	nothing: its heartbeat and its finish are Postgres rows	Spring Batch's repository is JDBC: the same property	Celery's result backend on Redis: the result is gone
The upgrade never taken	a sentinel key to detect a flush and rebuild at once		rejected: the sweep is 5 s and a marker is a moving part for no visible gain

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Detecting the flush with a sentinel key was rejected.

A key written at boot and checked on every sweep would say "Redis was wiped" and let the worker rebuild at once instead of within one sweep. The sweep already runs every five seconds and already does the rebuild; the sentinel adds a write, a read and a branch for a gain nobody can see. The same reasoning killed a "queue depth" gauge in E3: measure what the row says.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The vacuity guards exist because a first draft passed with nothing waiting.

The script's runs were too short: by the time the wipe landed, every run had been claimed and none was in Redis's `wait` list, so "the waiting runs were offered again" was true of an empty set. §4a's rule is that a chaos check must prove the failure was *present*; the script now reads `llen bull:ai:wait` before the wipe and the judge refuses a round that saw zero.

The principle, in the words the epic keeps using: the row is the truth and the message is a pointer to it. A pointer can be lost and re-derived from the truth (the sweep); the truth cannot be re-derived from the pointer. Every place an earlier epic was tempted to keep state only in Redis (the lease's token, the job's lock, the presence) was made re-derivable or was made a hint, and E9.2b is the receipt.

5 · Progress through the row: steps that survive a reload and never show a dead attempt's work

A person who asked the AI for a page sees the nodes appear on the canvas, but not what the AI is *doing*: which tool it called, whether a call was refused, how far it is. F30 asks for a list of steps that streams while the run goes and survives a reload. The transport question came first and was answered with the one already there: the panel polls the run once a second (E3.2's status poll), so the steps ride the same request and reach the browser through the same `/api` path the ngrok host proxies. No SSE (a new long-lived connection through Vite's proxy and ngrok for a few events a second), no sync socket (the room is the document's channel, and a run's bookkeeping in it would be journaled, fenced and replayed with the ops).

FROM packages/contracts/src/index.ts

```
/**
 * F30: one tool call of a run, as the panel lists it. `tool` is one of OUR tool names; `detail` comes from the
 * MODEL's arguments (a component name, a prop key, a node id): untrusted text, shown as text, never as markup.
 */
export const RunStep = z.strictObject({ tool: FailureReason, ok: z.boolean(), detail: z.string().max(80) });
export type RunStep = z.infer<typeof RunStep>;
/** Keeps the last this-many steps: a progress view, not a transcript (SPEC §6: no persisted full transcript). */
export const MAX_RUN_STEPS = 50;
/** What a running `ai` job reports (jobs.output): the steps of its CURRENT attempt only. */
export const RunProgress = z.strictObject({ steps: z.array(RunStep).max(MAX_RUN_STEPS) });
export type RunProgress = z.infer<typeof RunProgress>;
```

`detail` is the model's own argument: a component name, a prop key, a node id. It is *untrusted text* in every sense the earlier lessons gave that phrase: the contract caps it at 80 characters, the worker strips every `\p{C}` character (control characters and, in particular, bidi overrides that could make a line read backwards on screen), the row parser reads it back through the same contract, and the panel renders it as a text node. `tool` reuses `FailureReason`'s shape, a lower-case name, because that is what a tool name is and it keeps the enum open for a worker newer than the browser.

FROM apps/worker/src/progress.ts

```
import { MAX_RUN_STEPS, type RunStep } from "@noon/contracts";
import type { AgentTool } from "../tools.ts";

// Which argument says what a call was about. The rest (props, index) is detail the panel does not need.
const SUBJECT: Partial<Record<string, string>> = { add_node: "component", set_prop: "key", move_node: "nodeId",
remove_node: "nodeId" };

/**
 * F30: one tool call as the panel shows it. The subject is the MODEL's own argument, so it is untrusted text: only a
 * string is taken, invisible and control characters go (a bidi override could make a line read as something else),
 * and it is cut to what the contract stores. The browser shows it as text.
 */
export function stepOf(tool: string, args: unknown, ok: boolean): RunStep {
  const key = SUBJECT[tool];
  const raw = key !== undefined && typeof args === "object" && args !== null && Object.hasOwn(args, key) ? (args as
Record<string, unknown>)[key] : undefined;
  return { tool, ok, detail: typeof raw === "string" ? raw.replace(/\p{C}/gu, "").slice(0, 80) : "" };
}

/**
 * The same tools, each call also added to the run's steps; `onSteps` gets the whole list (the last MAX_RUN_STEPS)
 * after every call, so a write that fails is made good by the next one.
 */
export function withProgress(tools: AgentTool[], onSteps: (steps: RunStep[]) => void): AgentTool[] {
  const steps: RunStep[] = [];
  return tools.map((t) => ({
    ...t,
    run: async (args) => {
      const result = await t.run(args);
      steps.push(stepOf(t.name, args, result.ok));
      if (steps.length > MAX_RUN_STEPS) steps.shift();
      onSteps([...steps]);
      return result;
    },
  }));
}
```

FROM apps/worker/src/ai.ts

```
// One write at a time, in order: a later list never lands before an earlier one. A failed write is left: the next
carries every step.
let reporting = Promise.resolve();
const tools = withProgress(buildTools(peer, manifest, ids.nodeId), (steps) => { reporting = reporting.then(() =>
report(job, { steps })).catch(() => undefined); });
```

FROM apps/worker/src/ai.ts

```
    } finally {
      clearInterval(watchdog);
      abort.abort(); // stops the model, and removes the listener on 'stopping'
      peer.close(); // an op still waiting is answered 'connection_closed'; everything the agent was told succeeded, did
      await reporting; // before the worker finishes the row: a step written after that would land nowhere (report needs
        'running')
    }
```

Three decisions in the worker. **The whole list, every time:** `onSteps` gets the last fifty steps after each call, so a write that failed is made good by the next one and no step is ever lost to one dropped query. **One write at a time, in order:** `reporting = reporting.then(...)` chains the writes, so a later list never lands before an earlier one, and a list written at step 7 cannot be overwritten by the list of step 6 arriving late. **Awaited before the row is finished:** `report` needs `status = 'running'` (section 2's rule that a finished job is not written to), so the handler's `finally` waits for the chain before returning to the worker, which will call `finish`.

FROM packages/db/src/index.ts

```
    async report({ queue, jobId, orgId, attempt }, output) {
      const valid = (queue === "ship" ? ShipOutput : queue === "ai" ? RunProgress :
        PreviewOutput).nullable().parse(output); // the contract the reader will use, BEFORE the write
      if (!isId(jobId) || !isId(orgId)) return;
      await pool.query("update jobs set output = $3 where org_id = $1 and id = $2 and status = 'running' and ($4::int is
        null or attempts = $4)", [orgId, jobId, JSON.stringify(valid), attempt ?? null]);
    },
```

`report` is fenced by the attempt like every other write, and here the fence has a visible consequence: a slow dead attempt's steps *land nowhere*, so the panel never lists a tool call nobody is making. And `requeueStale` clears an AI job's `output` when it requeues it, so the retry's list starts empty rather than showing the dead attempt's steps as if they were its own. The integration test does both: reports from attempt 1, requeues, asserts an empty list, claims attempt 2, reports from attempt 1 again (the slow worker waking up), asserts still empty, reports from attempt 2, asserts that list.

FROM packages/db/src/run-progress.int.test.ts

```
test("a dead attempt's steps never show: the requeue forgets them and its late report lands nowhere (F28 fence)", async () => {
  const jobs = t.db.jobStore();
  const { run, doc, key } = await aRun();
  await jobs.claim(key);
  await jobs.report({ ...key, attempt: 1 }, { steps: [step("FromTheDeadOne")] });
  await t.rawQuery("update jobs set heartbeat_at = now() - interval '1 minute' where id = $1", [run.id]);
  await jobs.requeueStale(10_000, 3);
  expect((await t.db.forOrg(doc.orgId).getRun(doc.id, run.id)?.steps).toEqual([]);
  await jobs.claim(key); // attempt 2
  await jobs.report({ ...key, attempt: 1 }, { steps: [step("FromTheDeadOne")] }); // the slow first worker wakes up
  expect((await t.db.forOrg(doc.orgId).getRun(doc.id, run.id)?.steps).toEqual([]);
  await jobs.report({ ...key, attempt: 2 }, { steps: [step("Card")] });
  expect((await t.db.forOrg(doc.orgId).getRun(doc.id, run.id)?.steps).toEqual([step("Card")]));
});
```

FROM packages/db/src/index.ts

```
// A run's progress is written by our own worker but quotes the model: read as untrusted, and a row that does not parse
shows no steps.
const RunRow = z
  .object({ id: z.string(), org_id: z.string(), document_id: z.string(), status: z.string(), input: z.object({
    instruction: z.string() }), error: z.string().nullable(), output: z.unknown(), created_at: timestamp, started_at:
    nullableTimestamp, finished_at: nullableTimestamp })
  .transform((r): Run => {
    const progress = RunProgress.safeParse(r.output);
    return Run.parse({ id: r.id, orgId: r.org_id, documentId: r.document_id, status: r.status, instruction:
      r.input.instruction, error: r.error, createdAt: r.created_at, startedAt: r.started_at, finishedAt: r.finished_at, steps:
      progress.success ? progress.data.steps : [] });
  });
```

`RunRow` parses `output` with `RunProgress.safeParse` and shows *no steps* for a row that does not parse, rather than throwing. The row was written by our own worker through the same contract, so a failure here means someone edited the column by hand or a newer worker wrote a shape this build does not know; either way the run must still be readable, and its steps are the part to drop. The test writes `{"tool": "<b>", "ok": 1}` straight into the column and asks for an empty list.

FROM `apps/api/src/app.ts`

```
// F30: the newest run, with its steps so far. A page that reloads mid-run asks this and carries on polling that run.
// ponytail: progress rides the 1 s poll the panel already makes (no SSE, nothing new through the ngrok host). A
// second of lag per step; a push channel if a run's steps ever need to arrive faster than that.
document.get("/run", need("viewer"), async (c) => {
  const run = await db.forOrg(c.var.doc.orgId).getLatestRun(c.var.doc.id);
  return run === undefined ? notFound(c) : c.json(DocumentRun.parse({ run }));
});
```

FROM `apps/web/src/AiPanel.tsx`

```
// The newest run, after a reload too (F30). A run started before this answers is not replaced by an older one.
useEffect(() => { readLatestRun(documentId).then((latest) => { if (latest) setRun((current) => current ?? latest); }, ()
=> undefined); }, [documentId]);
useEffect(() => {
  if (!running) return;
  const timer = setInterval(() => { readRun(run).then(setRun, () => undefined); }, 1000); // a failed look is tried
  again in a second
  return () => { clearInterval(timer); };
}, [running, run]);
```

The reload. `GET /documents/:id/run` answers the document's newest run (as `GET /ship` does), viewer role, 404 when the document is not the caller's, `null` when the AI was never asked. The panel reads it on mount; `setRun((current) => current ?? latest)` is the guard the comment names: a run the person started in the half-second before the answer arrived must not be replaced by an older one, and a functional `useState` is how React lets you decide against the value that is current *now* rather than the one captured when the effect started (lesson 2's external-store idiom, in one line). Steps stay on the row after the run ends, so a reload afterwards still says what the run did.

FROM `apps/web/src/progress.ts`

```
import type { RunStep } from "@noon/contracts";

// Our tools, in the user's words. The list is OPEN (a newer worker may have a tool this build does not know): an
// unknown one is shown by its name, never dropped. A Map: a tool named "constructor" must find nothing, not
Object.prototype's.
const DOING = new Map(Object.entries({
  read_tree: "Read the page",
  read_manifest: "Read the design system",
  add_node: "Added",
  set_prop: "Set",
  move_node: "Moved",
  remove_node: "Removed",
})));

/** F30: one step of a run as a line of TEXT. `detail` is the model's own words: React renders it as text, never markup.
*/
export function stepText(step: RunStep): string {
  const line = [DOING.get(step.tool) ?? step.tool, step.detail].filter((part) => part !== "").join(" ");
  return step.ok ? line : `${line}: not applied`;
}
```

FROM `e2e/progress.spec.ts`

```
await page.reload();
await expect(page.getByRole("status")).toHaveText("live");
// The same run, still running, with every step it had taken: none lost, none from anywhere else.
await expect(page.locator("#ai-status")).toHaveAttribute("data-run-status", "running");
await expect.poll(() => steps.count()).toBeGreaterThanOrEqual(before);
await expect(steps.first()).toHaveText("Added Card");
await expect(page.getByRole("button", { name: "Cancel the AI run", exact: true })).toBeVisible(); // and it can still be
```

<pre>stopped await expect(page.getByRole("button", { name: "Ask the AI", exact: true })).toBeDisabled();</pre>		
TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>raw.replace(/\p{C}/gu, "").slice(0, 80) :</code> Unicode property escapes	<code>s.replaceAll("\\p{C}", "")</code>	<code>"".join(c for c in s if unicodedata.category(c)[0] != "C")</code>
<code>reporting = reporting.then(() => ...).catch(() => undefined) :</code> a promise chain as a serial queue	a single-thread <code>Executor</code>	an <code>asyncio.Queue</code> with one consumer
<code>setRun((current) => current ?? latest) :</code> a functional state update	no analogue; a compare-and-set on an <code>AtomicReference</code> is the nearest	no analogue
<code>RunProgress.safeParse(r.output)</code> and <code>steps: []</code> on failure	a <code>try</code> around Jackson's <code>readValue</code>	<code>pydantic.ValidationError</code> caught and an empty list
<code>Object.hasOwn(args, key)</code> before reading a model's argument	<code>map.containsKey</code>	<code>key in args</code>
<code>{run.steps.map((step, at) => <li key={at}> {stepText(step)})} :</code> text nodes only	Thymeleaf's <code>th:text</code> escapes; <code>th:utext</code> would not	Django templates autoescape; <code> safe</code> would not

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

SSE and the sync socket were rejected as the transport.

Server-sent events would stream each step at once, and they would add a long-lived connection through two proxies (Vite's and ngrok's) for something that arrives a few times a second at most. The sync socket already exists and is the wrong channel: everything on it is an op or presence, journaled, fenced by the lease, replayed on reconnect; a run's steps are none of those things. The 1 s poll already existed for the status and now carries the steps; a step shows up to a second late, which the ponytail names.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The list is reported whole, not appended.

Appending one step per write is the obvious shape, and it makes every dropped write a hole in the list and every reordered pair a list out of order. Sending the last fifty every time makes a write idempotent (the same list twice is the same list) and self-healing (the next write carries what the last one lost), at the cost of a few kilobytes per tool call, which a run makes a few times a second at most.

The principle: a progress view is derived state, so put it where the state is and let the readers you already have read it. The row already had a poller and already had a fence; the steps inherit both, and the one new route exists for the one new question, "which run should a fresh page follow".

6 · Rate limits in Postgres: one upsert, inside the transaction, with the database's clock

An org that starts too many AI runs costs money and holds worker slots; F31 asks for a per-org limit with a 429 that says *exactly* how long to wait. The first design question is where to count, and the answer decides everything else. A counter in each api process lets N instances allow N times the limit. Redis `INCR` with `EXPIRE` is shared, and it was rejected for two reasons the commit names: it is a second store that can fail on its own, so the code must pick between failing open and failing closed when Redis is away; and it cannot join the run's transaction, so a run that was refused as busy, or rolled back, or replayed, would have been charged, and the code would need a compensating decrement. Counting in Postgres, in the same transaction as the run, makes the count exactly the runs that exist.

FROM packages/db/migrations/0020_rate_limits.sql

```
-- E9.5 (F31): rate limits, counted HERE so every api instance shares one count (a counter in each process would let
-- N instances allow N times the limit). One row per key ('ai_run:<org id>', and E9.6's per user and per address keys):
-- the fixed window it is counting (seconds since the epoch divided by the window's length) and the hits in it. A hit
-- is ONE upsert, so two at the same moment take turns on the row, and a hit made inside a transaction that rolls back
-- (a run refused as busy, or over the limit) is not counted. No foreign key: a key names whatever it limits.
create table rate_limits (
  key text primary key check (length(key) between 1 and 200),
  win bigint not null,
  hits integer not null check (hits >= 1)
);

-- F31: the usage view sums an org's usage per user.
create index usage_org_user on usage (org_id, user_id);
```

FROM packages/db/src/limit.ts

```
import { z } from "zod";

/**
 * E9.5 (F31): a rate limit is 'limit' hits per key in each fixed window of 'windowSeconds', counted in Postgres (the
 * rate_limits table, migration 0020) so every api instance shares one count. Reused by E9.6 (per user, per address,
 * session minting) through 'Db.take'. ponytail: fixed windows, so a key may spend 'limit' at the end of one window and
 * 'limit' again at the start of the next (twice the rate for a moment); upgrade: a sliding window (two counters,
 * weighted) if that burst ever matters.
 */
export const Rule = z.strictObject({ limit: z.number().int().min(1), windowSeconds: z.number().int().min(1).max(86_400)
});
export type Rule = z.infer<typeof Rule>;
export type Verdict = { ok: true } | { ok: false; retryAfterSeconds: number };

/**
 * 'hits': this key's count in 'window' (the window's index since the epoch), this hit included; 'now': the DATABASE's
 * clock in seconds, so api instances whose clocks disagree still give one answer. Over the limit, the wait is until the
 * window ends, rounded up (a client that waits exactly that long is let in), and never under one second.
 */
export function verdict({ hits, window, now }: { hits: number; window: number; now: number }, rule: Rule): Verdict {
  if (hits <= rule.limit) return { ok: true };
  return { ok: false, retryAfterSeconds: Math.max(1, Math.ceil((window + 1) * rule.windowSeconds - now)) };
}
```

A *fixed window*: time is cut into windows of `windowSeconds`, numbered since the epoch, and a key may be hit `limit` times per window. `verdict` is pure and takes the database's clock as `now`, so two api instances whose clocks disagree still give one answer, and the wait is until the window ends, rounded *up* and never under one second: a client that waits exactly what it is told must be let in, and a "retry in 0 seconds" is a lie. The unit test pins the rounding at the boundary.

FROM packages/db/src/index.ts

```
/**
 * One hit on a rate limit (limit.ts). 'greatest': a hit that computed its window just before the boundary but reached
 * the row after one computed just past it must not move the row back a window (and reset its count).
 */
async function take(key: string, rule: Rule, via: Pool | PoolClient = pool): Promise<Verdict> {
  const { windowSeconds } = Rule.parse(rule);
  const hit = await one(
    z.object({ hits: z.number().int(), win: count, now: z.number() }),
    "insert into rate_limits as r (key, win, hits) values ($1, floor(extract(epoch from clock_timestamp())) /
$2)::bigint, 1) " +
    "on conflict (key) do update set hits = case when r.win >= excluded.win then r.hits + 1 else 1 end, win =
greatest(r.win, excluded.win) " +
    "returning hits, win::text, extract(epoch from clock_timestamp())::float8 as now",
    [z.string().min(1).max(200).parse(key), windowSeconds],
    via,
  );
  if (!hit) throw new Error("rate limit: no row");
  return verdict({ hits: hit.hits, window: hit.win, now: hit.now }, rule);
}
```

One upsert. `insert ... on conflict (key) do update` is atomic per row: two hits at the same moment take turns on the row lock, and each sees the other's count. Read the `greatest(r.win, excluded.win)`: a hit that computed its window just before the boundary (`floor(epoch / windowSeconds)` is evaluated when the statement starts) but reached the row after a hit computed just past it must not move the row *back* a window and reset its count to 1, which would hand out a fresh limit. The `case` counts it into the newer window instead. `clock_timestamp()` rather than `now()`, because `now()` is the transaction's start time and this hit may be several statements into one.

Inside the transaction. Section 1's `createRun` cut shows the order: `take` first, then `insertRun`, in one transaction that commits only if a run was made. So a busy document (409), a vanished one (404), a refused run and, through `withKey`, a replayed key all cost nothing; and a request refused over the limit rolls back its idempotency claim too, so the retry after `Retry-After` may use the same key, which the integration test asserts by sleeping exactly what the header said. The hit holds the org's counter row until the insert is done, so an org's run starts take turns for the length of one insert, which is the price of exactness and a small one.

FROM `apps/api/src/app.ts`

```
/** F31: over a rate limit. The wait is in the Retry-After header (RFC 9110, whole seconds) and, the same number, in the body. */
const limited = (c: Context, retryAfterSeconds: number, error: "rate_limited" | "too_many_attempts" = "rate_limited") => {
  c.header("retry-after", String(retryAfterSeconds));
  return c.json({ error, retryAfterSeconds } satisfies ErrorBody, 429);
};
```

FROM `packages/contracts/src/index.ts`

```
/** With 429 rate_limited and too_many_attempts (F31): the same number as the Retry-After header, for a client that reads only the body. */
retryAfterSeconds: z.number().int().min(1).optional(),
```

FROM `apps/api/src/config.ts`

```
/** F31: the default AI run limit, per org. ponytail: one limit for every org; upgrade: a column on orgs when plans differ. */
export const AI_RUN_LIMIT: Rule = { limit: 60, windowSeconds: 3600 };
/**
 * E9.6 (F31): every HTTP route but the probes and Gitea's webhook. `user`: any request a caller is known for (an open
 * editor polls about 3 a second). `address`: the routes that name no user (sign-up, sign-in, sign-out, /auth/me) and
 * any request whose caller is not recognised, per client address. `mint`: POST /documents/:id/session, per user,
 * tighter than `user` and counted before the document is looked up, so a revoked collaborator's peer that keeps
 * asking is refused cheaply. `attempt`: sign-up and sign-in per email, so an online guesser gets 10 tries per 5 minutes
 * on one account whatever address they come from. ponytail: constants, one for everyone; upgrade: env or a plan column.
 */
export const HTTP_LIMITS = {
  user: { limit: 600, windowSeconds: 60 },
  address: { limit: 300, windowSeconds: 60 },
  mint: { limit: 60, windowSeconds: 60 },
  attempt: { limit: 10, windowSeconds: 300 },
} satisfies Record<string, Rule>;
```

Failure mode, deliberately closed. The count lives where the run lives: with Postgres away no run can be made either, so the limit cannot fail open, and there is no second store whose absence would need a policy. That sentence is the one to say in an interview when someone proposes Redis for the counter: ask what the code does when Redis is away, and whether "allow everything" or "refuse everything" is the answer they want under load.

FROM `packages/db/src/limit.test.ts`

```
test("up to the limit goes ahead; the next one waits for the window to end, to the second", () => {
  const window = 480_000; // 3600 * 480000 s since the epoch: the window's first second
  const start = window * 3600;
  expect(verdict({ hits: 1, window, now: start }, hourly)).toEqual({ ok: true });
  expect(verdict({ hits: 3, window, now: start + 10 }, hourly)).toEqual({ ok: true });
  expect(verdict({ hits: 4, window, now: start + 10 }, hourly)).toEqual({ ok: false, retryAfterSeconds: 3590 });
  expect(verdict({ hits: 4, window, now: start + 3599.2 }, hourly)).toEqual({ ok: false, retryAfterSeconds: 1 }); //
  rounded UP: never "retry now" too early
```

```
expect(verdict({ hits: 4, window, now: start }, hourly)).toEqual({ ok: false, retryAfterSeconds: 3600 });
});
```

FROM `apps/api/src/rate-limit.int.test.ts`

```
test("ten starts at once across two api instances, limit three: three runs, seven 429s whose Retry-After is the window's end", async () => {
  const rule = { limit: 3, windowSeconds: 3600 };
  const [a, b] = [instance(rule), instance(rule)];
  const docs = await documents("ann@example.com", 10);
  const answers = await Promise.all(docs.map((doc, i) => start(i % 2 === 0 ? a : b, "ann@example.com", doc)));
  const left = await secondsLeft(rule.windowSeconds);
  expect(answers.map((res) => res.status).sort()).toEqual([201, 201, 201, ...Array<number>(7).fill(429)]);
  expect(await runsOf(docs[0]?.orgId ?? "").toBe(3));
  for (const res of answers.filter((each) => each.status === 429)) {
    const retry = Number(res.headers.get("retry-after"));
    expect(Number.isInteger(retry)).toBe(true);
    expect(Math.abs(retry - left)).toBeLessThanOrEqual(2); // measured a moment later, rounded up
    expect(ErrorBody.parse(await res.json())).toEqual({ error: "rate_limited", retryAfterSeconds: retry });
  }
  // Another org has its own count.
  const [theirs] = await documents("bob@example.com", 1);
  expect((await start(a, "bob@example.com", theirs as Document)).status).toBe(201);
});
```

FROM `apps/api/src/rate-limit.int.test.ts`

```
test("only runs that are made count: a busy document and a refused run cost nothing, and a replayed key is answered, not refused", async () => {
  const app = instance({ limit: 2, windowSeconds: 3600 });
  const [one, two, three] = await documents("cat@example.com", 3) as [Document, Document, Document];
  const key = crypto.randomUUID();
  const first = await start(app, "cat@example.com", one, key);
  expect(first.status).toBe(201);
  const made = Run.parse(await first.json());
  // `one` is busy: 409, and not a hit, so `two` still fits.
  expect((await start(app, "cat@example.com", one)).status).toBe(409);
  expect((await start(app, "cat@example.com", two)).status).toBe(201);
  expect((await start(app, "cat@example.com", three)).status).toBe(429);
  expect((await start(app, "cat@example.com", three)).status).toBe(429); // the refusals did not add up to a longer wait
  // Over the limit, the first request's retry still gets the run it made.
  const replay = await start(app, "cat@example.com", one, key);
  expect(replay.status).toBe(201);
  expect(Run.parse(await replay.json()).id).toBe(made.id);
  expect(await runsOf(one.orgId)).toBe(2);
});
```

FROM `packages/db/src/usage.int.test.ts`

```
test("take: twenty hits at once on one key, limit five: exactly five go ahead, and the rest are told the same window end", async () => {
  const rule = { limit: 5, windowSeconds: 3600 };
  const key = `test:${crypto.randomUUID()}`;
  const verdicts = await Promise.all(Array.from({ length: 20 }, () => t.db.take(key, rule)));
  expect(verdicts.filter((v) => v.ok)).toHaveLength(5);
  const waits = new Set(verdicts.flatMap((v) => (v.ok ? [] : [v.retryAfterSeconds])));
  expect(Math.max(...waits) - Math.min(...waits)).toBeLessThanOrEqual(1);
  expect(Math.max(...waits)).toBeLessThanOrEqual(3600);
  expect(await t.db.take(`test:${crypto.randomUUID()}`, rule)).toEqual({ ok: true }); // another key, its own count
});
```

Fixed, sliding, or a bucket. The ponytail is named in `limit.ts`: a key may spend its limit at the end of one window and again at the start of the next, twice the rate for a moment. The table is the comparison an interviewer wants, with the honest reason this repo chose the simplest.

ALGORITHM	STATE	BEHAVIOUR	HERE
Fixed window (this repo)	one row per key: window index and count; one upsert	up to 2× the limit across a boundary; the wait is exact and cheap to compute	sixty runs an hour, and a burst of 120 across the hour mark is a nuisance, not a cost worth a second counter
Sliding window (two counters, weighted)	the current and previous window's counts; the previous weighted by how far into the current window we are	no boundary burst; approximate (assumes an even spread in the previous window)	the named upgrade: one more column, the same upsert
Sliding log	every hit's timestamp; count those within the last window	exact; O(hits) storage and a delete per hit	a table per key would grow with the traffic it limits
Token bucket	a level and a last-refill time; hits take a token, tokens refill at a rate	allows bursts up to the bucket size, then a steady rate; the classic API shape (AWS, Stripe)	the right choice for "600 requests a minute but bursts are fine"; two columns and a little arithmetic in the upsert
Leaky bucket	a queue drained at a fixed rate	smooths bursts into a steady stream; adds latency rather than refusing	a queue, not a limiter: the jobs table already is one

FROM packages/db/src/index.ts

```
usage: async (input) => {
  const none = { runs: 0, inputTokens: 0, outputTokens: 0, cacheReadTokens: 0, cacheWriteTokens: 0, costUsd: 0 };
  if (!orgExists) return { totals: none, byUser: [], byDay: [], items: [], nextCursor: null };
  // One REPEATABLE READ snapshot for every part (the E3.4 finding): a run recorded between two of these reads
  // would otherwise show in the totals and not in the items. Read only: nothing here may write.
  return inTx(async (client) => {
    // The user's email as it is now; a deleted user's runs keep their cost, and show no one.
    const items = await page(UsageRow, "(select usage.*, users.email from usage left join users on users.id =
usage.user_id t", "t.org_id = $1", [orgId], input, true, client);
    if (!items) return undefined;
    const totals = await one(UsageTotalsRow, `select ${USAGE_SUMS} from usage where org_id = $1`, [orgId],
client);
    const byUser = await rows(
      UsageUserRow,
      `select s.*, users.email from (select user_id, ${USAGE_SUMS}, sum(cost_usd) as cost from usage where org_id
= $1 group by user_id) s ` +
      "left join users on users.id = s.user_id order by s.cost desc, users.email nulls last limit $2",
      [orgId, USAGE_USERS],
      client,
    );
    const byDay = await rows(
      UsageDayRow,
      `select to_char(created_at at time zone 'UTC', 'YYYY-MM-DD') as day, ${USAGE_SUMS} from usage where org_id =
$1 group by 1 order by 1 desc limit $2`,
      [orgId, USAGE_DAYS],
      client,
    );
    return { totals: totals ?? none, byUser, byDay, ...items };
  }, () => true, "begin transaction isolation level repeatable read read only");
},
```

Usage, the other half of F31. The owner reads tokens and estimated cost per run, per person and per day, and the E3.4 review's finding is honoured in the first line: all four reads happen in one `REPEATABLE READ`, read-only transaction, so a run recorded between two of them cannot show in the totals and not in the items. The user's email is joined as it is *now*, and a deleted user's runs keep their cost and name no one. The web view is tables of text, with a caption and row and column headers, and the AI panel turns a 429 into a sentence with `waitWords`, rounded up for the same reason the verdict is.

FROM apps/web/src/usage.ts

```
/** How long to wait, in words, rounded UP (a person told "1 minute" who comes back at 61 seconds must not be refused
again). */
```

```
export function waitWords(seconds: number): string {
  if (seconds < 60) return seconds === 1 ? "1 second" : `${String(seconds)} seconds`;
  const minutes = Math.ceil(seconds / 60);
  return minutes === 1 ? "1 minute" : `${String(minutes)} minutes`;
}
```

FROM `apps/web/src/api.ts`

```
/** F31: "not now": over a rate limit. Asking again after `retryAfterSeconds` may work; asking sooner will not. */
class Limited extends Error {
  readonly retryAfterSeconds: number;
  constructor(message: string, retryAfterSeconds: number) {
    super(message);
    this.retryAfterSeconds = retryAfterSeconds;
  }
}
/** Retry-After as whole seconds (at least one); a missing or odd header reads as a minute. */
const retryAfter = (res: Response): number => {
  const seconds = Number(res.headers.get("retry-after"));
  return Number.isInteger(seconds) && seconds >= 1 ? seconds : 60;
};
```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

`insert ... on conflict
(key) do update set
hits = case ... end : an
upsert with arithmetic`

the same SQL; `MERGE` on other databases

the same SQL

`extract(epoch from
clock_timestamp()) : the
database's clock, per
statement`

`CURRENT_TIMESTAMP` is the transaction's; use the
same function

the same

`Math.ceil((window + 1)
* windowSeconds - now)`

`Math.ceil` on double s

`math.ceil`

`z.strictObject({
limit, windowSeconds
}) with .max(86_400)`

a record with Bean Validation `@Max`

a pydantic model with `le=86400`

`via: Pool | PoolClient
= pool : the same query
inside or outside a
transaction`

a method taking a `Connection`, with an
overload that borrows one

a function taking a cursor

`c.header("retry-
after",
String(seconds))`

`ResponseEntity.status(429).header("Retry-
After", ...)`

`HttpResponse(status=429, headers={"Retry-
After": ...})`

`begin transaction
isolation level
repeatable read read
only`

`@Transactional(isolation =
REPEATABLE_READ, readOnly = true)`

`connection.set_session(isolation_level=...,
readonly=True)`

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Redis INCR + EXPIRE was rejected.

It is the textbook shared counter and it fails the two questions this design asks. When Redis is away, what does the code do: open (every org unlimited under exactly the load that took Redis down) or closed (no runs, with Postgres healthy)? And when a run is refused, rolled back or replayed, how is the count corrected: a DECR that itself can fail? Counting in the run's transaction has no second failure and no compensation.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The greatest(win) clause exists because of a hit that crossed the boundary in flight.

The first upsert wrote `win = excluded.win` and reset the count when the windows differed. Two hits at 3599.99 s and 3600.01 s into an hour could land in the other order (the first is evaluated, waits on the row lock, and reaches it after the second): the row moved back to the old window with `hits = 1`, and the next hit in the new window saw `r.win < excluded.win` and reset again. `greatest` and the `>=` in the `case` make the row's window monotonic.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A single limit for every org is the ponytail; a column is the upgrade.

`AI_RUNS_PER_HOUR` is one number for the deployment. The moment plans differ, the rule comes from a column on `orgs` read in the same transaction; nothing in `take` changes. Fixed windows are the other named ceiling: a burst across the hour mark is twice the limit for a moment, which for sixty runs an hour is not a cost worth a second counter.

The principle: count where the counted thing is written, in the same transaction, and take the clock from the same place. Then "the count is exactly the runs that exist" is a fact rather than a reconciliation, the limit cannot fail open because there is no second store to lose, and the wait you tell the client is the truth to the second.

7 · Every route limited, and who is asking: per user, per address, and the header you may not believe

E9.6 spreads section 6's upsert over every HTTP route, and the question becomes *which key*. A caller the api recognises is charged per user, whatever address they come from: an editor's open canvas polls about three times a second, so 600 a minute. A public route (sign-up, sign-in, sign-out, `/auth/me`) names no user, and a request whose cookie is bad or missing is nobody, so both are charged per client address: 300 a minute, and that is what turns a 401 into a 429 for someone guessing session tokens. Minting a sync session has a tighter count of its own, and sign-in a count per email that E8.1 left a seam for.

FROM `apps/api/src/app.ts`

```
// E9.6 (F31): and every route is rate limited but the probes (an orchestrator refused a probe kills a healthy
// process) and Gitea's webhook (the HMAC is its gate, and a refused delivery would only be lost to the reconcile).
// A known caller is counted per user; a public route, or a caller nobody recognises, per client address
// (client-address.ts: X-Forwarded-For only from a trusted proxy). Db.take is one upsert, so every api instance
// shares the count. ponytail: one Postgres write per request (and a hot row per user, held for one statement);
// ceiling: a few thousand requests a second on one primary; upgrade: Redis INCR, failing open. With Postgres away the
// take throws and the answer is 500, closed: every one of these routes needs the database anyway.
const UNLIMITED = new Set(["/health", "/ready", WEBHOOK_PATH]);
const PUBLIC_PATHS = new Set(["/health", "/ready", WEBHOOK_PATH, "/auth/signup", "/auth/signin", "/auth/signout",
"/auth/me"]);
// ponytail: an IPv6 address is its own key; ceiling: a client with a /64 has 2^64 of them; upgrade: key IPv6 by /64.
const byAddress = (c: Context) => db.take(`address:${clientAddress(peerOf(c), c.req.header("x-forwarded-for"),
trustProxy) ?? "unknown"}`, limits.address);
const requireUser = createMiddleware<{ Variables: { user: User } }>(async (c, next) => {
  if (UNLIMITED.has(c.req.path)) return next();
  if (PUBLIC_PATHS.has(c.req.path)) {
```

```

    const verdict = await byAddress(c);
    return verdict.ok ? next() : limited(c, verdict.retryAfterSeconds);
  }
  const user = await identify(c, db);
  const verdict = await (user ? db.take(`user:${user.id}`, limits.user) : byAddress(c));
  if (!verdict.ok) return limited(c, verdict.retryAfterSeconds);
  if (!user) return fail(c, 401, "unauthenticated");
  c.set("user", user);
  await next();
});

```

Read the order inside `requireUser`. The probes are never counted: an orchestrator that is refused a health check kills a healthy process, and a rate limit that can take the api down is a self-inflicted outage. The webhook is never counted: the HMAC is its gate, and a refused delivery would only fall back to the reconcile. For everyone else the count is taken *before* the 401 is answered, so an unauthenticated flood costs the flooders, and the answer over the limit is 429 whether or not they had a cookie: refusing first tells a guesser nothing about which of their guesses were closer.

FROM `apps/api/src/config.ts`

```

/** F31: the default AI run limit, per org. ponytail: one limit for every org; upgrade: a column on orgs when plans differ. */
export const AI_RUN_LIMIT: Rule = { limit: 60, windowSeconds: 3600 };
/**
 * E9.6 (F31): every HTTP route but the probes and Gitea's webhook. `user`: any request a caller is known for (an open
 * editor polls about 3 a second). `address`: the routes that name no user (sign-up, sign-in, sign-out, /auth/me) and
 * any request whose caller is not recognised, per client address. `mint`: POST /documents/:id/session, per user,
 * tighter than `user` and counted before the document is looked up, so a revoked collaborator's peer that keeps
 * asking is refused cheaply. `attempt`: sign-up and sign-in per email, so an online guesser gets 10 tries per 5 minutes
 * on one account whatever address they come from. ponytail: constants, one for everyone; upgrade: env or a plan column.
 */
export const HTTP_LIMITS = {
  user: { limit: 600, windowSeconds: 60 },
  address: { limit: 300, windowSeconds: 60 },
  mint: { limit: 60, windowSeconds: 60 },
  attempt: { limit: 10, windowSeconds: 300 },
} satisfies Record<string, Rule>;

```

FROM `apps/api/src/app.ts`

```

// E9.6: minting a sync session has its own, tighter limit per user, taken BEFORE the document is looked up. A
// collaborator whose share was revoked gets 404 here; a peer that keeps asking regardless is soon refused with a
// Retry-After (and a real client backs off), on top of its per-user limit.
app.use("/documents/:id/session", createMiddleware<{ Variables: { user: User } }>(async (c, next) => {
  if (c.req.method !== "POST") return next();
  const verdict = await db.take(`mint:${c.var.user.id}`, limits.mint);
  return verdict.ok ? next() : limited(c, verdict.retryAfterSeconds);
}));

```

The mint limit is taken *before the document is looked up*, and the reason is lesson 8's revoked collaborator: their peer's `/session` gets a 404 and gives up, but a client that keeps asking regardless (a bug, a script) would cost a membership read per request for a person who has no business with the document. Charging first makes the refusal cheap, and the tighter count (60 a minute against 600) means the guest is refused with requests to spare on their general count while the owner's mints are untouched.

FROM `apps/api/src/app.ts`

```

const attempt = allowAttempt ?? ((key: string) => db.take(`attempt:${createHash("sha256").update(key).digest("hex")}`, limits.attempt));

```

FROM `apps/api/src/app.ts`

```

// No such email and the wrong password are one answer, in one time: both cost exactly one script.
app.post("/auth/signin", async (c) => {
  const { email, password } = await body(c, SignInBody);
  const allowed = await attempt(`signin:${email.toLowerCase()}`);
  if (!allowed.ok) return limited(c, allowed.retryAfterSeconds, "too_many_attempts");
  const found = await db.credentialsFor(email);
  const matches = await verifyPassword(password, found?.passwordHash ?? (await dummyHash()));

```

```
    return found && matches ? signIn(c, found.user, 200) : fail(c, 401, "invalid_credentials");
  });
```

The seam E8.1 left, `allowAttempt`, now defaults to the limiter: 10 tries per 5 minutes per route and email, so an online guesser gets ten attempts at one account whatever address they come from, and the answer is `too_many_attempts` with a `Retry-After` now. The key is hashed because an email can be longer than the `rate_limits` key allows and because the table need not hold addresses; SHA-256 keeps the key short and the email out of the row. This is the fourth layer of lesson 8's sign-in: slow hash, constant-time compare, dummy hash, and now a count.

FROM `apps/api/src/client-address.ts`

```
/**
 * E9.6 (F31): who is asking, for the per-address rate limit on routes that name no user. The address is the TCP peer,
 * unless the peer is a proxy we trust, in which case it is the rightmost X-Forwarded-For entry that is not itself a
 * trusted proxy. Rightmost, not leftmost: every hop APPENDS what it saw (ngrok the visitor, http-proxy the ngrok
 * agent), so everything left of the last trusted hop's entry was written by the client and may say anything. A
 * header from a peer we do not trust is ignored entirely: anyone can send one.
 *
 * Who is trusted is TRUST_PROXY (config.ts): "loopback" by default, which is the Vite dev server (and the ngrok agent
 * behind it) when the api runs on the host. In compose the api sees the host's Vite through Docker's gateway, a
 * private address, so docker-compose.yml says "loopback,private": its port is published on 127.0.0.1 only, and the
 * sandboxes live on their own network, so a private peer is the host or one of our own containers.
 */
export type Trusted = (address: string) => boolean;
```

The header you may not believe. `X-Forwarded-For` is a comma-separated list, and the rule is that each proxy *appends* the address it saw the request come from. So the rightmost entry was written by the last proxy, the one before it by the proxy before, and the leftmost by whoever started the chain, which may be the client, who may write anything. The classic bug reads the *leftmost* entry as the client ("the original IP"): every guesser then picks a fresh address per request and the per-address limit counts nothing. The correct read is from the *right*: skip every entry that is itself a proxy you trust, and the first entry that is not is the client, vouched for by a trusted hop. And the whole header is believed only when the TCP peer itself is a trusted proxy; from anyone else it is ignored, because anyone can send one.

FROM `apps/api/src/client-address.ts`

```
/** `spec`: comma-separated "loopback", "private", addresses and CIDRs. Anything else throws: a typo must not trust the
world, or no one. */
export function trustedProxies(spec: string): Trusted {
  const list = new BlockList();
  for (const entry of spec.split(",").map((each) => each.trim()).filter((each) => each !== "")) {
    const named = NAMED[entry];
    if (named) {
      for (const [net, prefix] of named) list.addSubnet(net, prefix, family(net));
      continue;
    }
    const [net = "", prefix, extra] = entry.split("/");
    const bits = isIP(net) === 6 ? 128 : 32;
    if (isIP(net) === 0 || extra !== undefined || (prefix !== undefined && !(/^d{1,3}$/.test(prefix) && Number(prefix) <=
bits))) {
      throw new Error(`not an address, a CIDR, "loopback" or "private": ${entry}`);
    }
    if (prefix === undefined) list.addAddress(net, family(net));
    else list.addSubnet(net, Number(prefix), family(net));
  }
  return (address) => {
    const plain = unmap(address);
    return isIP(plain) !== 0 && list.check(plain, family(plain));
  };
}
```

FROM `apps/api/src/client-address.ts`

```
/** The client's address (see above), or undefined when there is no socket to ask (a request made in-process, in tests).
*/
export function clientAddress(peer: string | undefined, forwardedFor: string | undefined, trusted: Trusted): string |
undefined {
  if (peer === undefined) return undefined;
```

```

let client = unmap(peer);
if (!trusted(client) || forwardedFor === undefined) return client;
const hops = forwardedFor.split(",");
for (let i = hops.length - 1; i >= 0; i--) {
  const hop = unmap((hops[i] ?? "").trim());
  // Not an address: stop at the last one a trusted hop vouched for (a garbled header costs its sender, not others).
  if (isIP(hop) === 0) return client;
  client = hop;
  if (!trusted(hop)) return hop;
}
return client; // every hop a trusted proxy: the furthest one
}

```

Two more decisions in the code. `unmap` turns `::ffff:127.0.0.1`, an IPv4 client on a dual-stack socket, into `127.0.0.1`, so one client is one key and the loopback check works whichever family the socket reported. And a garbled entry (`1.2.3.4:80`, a word) stops the walk and charges the last address a trusted hop vouched for: a client who sends nonsense costs themselves, never someone else. `trustedProxies` refuses a spec it does not understand rather than trusting nobody or everybody: a typo in `TRUST_PROXY` must not start the api.

FROM `apps/api/src/config.ts`

```

// E9.6: the peers whose X-Forwarded-For names the client (client-address.ts). Unset or empty: loopback only.
TRUST_PROXY: z
  .string()
  .optional()
  .transform((value, ctx) => {
    try {
      return trustedProxies(value === undefined || value === "" ? "loopback" : value);
    } catch (err) {
      ctx.addIssue({ code: "custom", message: `TRUST_PROXY: ${err as Error}.message` });
      return z.NEVER;
    }
  }),

```

FROM `docker-compose.yml`

```

# The host's Vite proxy (and the ngrok agent behind it) reaches this container through Docker's gateway, a
# private address; the port below is bound to 127.0.0.1 and the sandboxes have their own network, so a private
# peer is the host or one of these services. The api run on the host needs only the default, loopback.
TRUST_PROXY: "loopback,private"

```

Why `loopback,private` in compose and `loopback` on the host: the api on the host sees Vite (and the ngrok agent behind it) on `127.0.0.1`; the api in a container sees the host's Vite through Docker's gateway, a private address. Trusting private addresses is safe there because the port is published on `127.0.0.1` only and the sandboxes live on their own network, so a private peer can only be the host or one of our own containers. The same string on a server with a public interface would be wrong, and the comment says so.

FROM `apps/api/src/client-address.test.ts`

```

test("behind a trusted proxy the client is the rightmost entry that is not a trusted proxy", () => {
  expect(clientAddress("127.0.0.1", "198.51.100.1", loopback)).toBe("198.51.100.1");
  // A visitor who writes their own header is to the LEFT of what ngrok appended: ignored.
  expect(clientAddress("127.0.0.1", "10.0.0.1, 1.2.3.4, 198.51.100.1", loopback)).toBe("198.51.100.1");
  // ngrok, then http-proxy (x fwd) appending the ngrok agent's loopback address.
  expect(clientAddress("::1", "198.51.100.1, 127.0.0.1", loopback)).toBe("198.51.100.1");
  expect(clientAddress("::ffff:127.0.0.1", "2001:db8::7", loopback)).toBe("2001:db8::7");
});

```

FROM `apps/api/src/http-limits.test.ts`

```

test("minting a session has its own tighter limit, charged before the document is looked up", async () => {
  const { call, charged, lookups } = app();
  expect(await call("/documents/d1/session", { method: "POST", headers: ann })).status().toBe(404); // not theirs (a
  revoked share)
  const refused = await call("/documents/d1/session", { method: "POST", headers: ann });
  expect(refused.status).toBe(429);
  expect(ErrorBody.parse(await refused.json())).toEqual({ error: "rate_limited", retryAfterSeconds: 7 });
  expect(lookups).toEqual(["d1"]); // the refused one never reached the lookup

```

```

    expect(charged).toEqual(["user:id-ann@example.com", "mint:id-ann@example.com", "user:id-ann@example.com", "mint:id-ann@example.com"]);
    // Other document routes are not minting.
    charged.length = 0;
    await call("/documents/d1/run", { headers: { "x-dev-user": "bob@example.com" } });
    expect(charged).toEqual(["user:id-bob@example.com"]);
  });

```

FROM apps/api/src/http-rate-limit.int.test.ts

```

test("public routes are counted per client address, and so is a caller nobody recognises", async () => {
  expect((await anonymous("/auth/me", "198.51.100.1")).status).toBe(200);
  expect((await anonymous("/orgs", "198.51.100.1")).status).toBe(401);
  expect((await anonymous("/auth/signout", "198.51.100.1", { method: "POST" })).status).toBe(204);
  await expectLimited(await anonymous("/auth/me", "198.51.100.1"));
  await expectLimited(await anonymous("/orgs", "198.51.100.1")); // guessing sessions is limited like the rest
  // Another client behind the same proxy is another address.
  expect((await anonymous("/auth/me", "198.51.100.2")).status).toBe(200);
  // What the client writes LEFT of the proxy's own entry is not believed: still 198.51.100.1.
  await expectLimited(await anonymous("/auth/me", "203.0.113.50, 198.51.100.1"));
});

```

FROM apps/api/src/http-rate-limit.int.test.ts

```

test("an untrusted peer's X-Forwarded-For is ignored: changing it does not buy a fresh count", async () => {
  // A real socket, so there IS a peer (loopback, trusted by nobody here).
  const server = await startServer({ port: 0, db: ctx.db.db, identify: devHeaderIdentity, sessions: TEST_SESSIONS,
    enqueue: () => Promise.resolve(), limits, trustProxy: trustedProxies("") });
  try {
    const statuses = [];
    for (const xff of ["192.0.2.1", "192.0.2.2", "192.0.2.3", "192.0.2.4"]) statuses.push((await
    fetch(`${server.url}/auth/me`, { headers: { "x-forwarded-for": xff } })).status);
    expect(statuses).toEqual([200, 200, 200, 429]);
  } finally {
    await server.close();
  }
});

```

FROM apps/api/src/http-rate-limit.int.test.ts

```

describe("integration:session-mint-limit", () => {
  test("a revoked collaborator who keeps minting is refused with a retry time, on a count of its own; the owner is untouched", async () => {
    // The mint count (2) is tighter than the per-user one (8): the guest's third mint is refused with requests to spare.
    const owner = "own@example.com";
    const guest = "guest@example.com";
    const org = Org.parse(await (await as(owner, "POST", "/orgs", { name: "Mint" })).json());
    const ws = Workspace.parse(await (await as(owner, "POST", `/orgs/${org.id}/workspaces`, { name: "w" })).json());
    const doc = Document.parse(await (await as(owner, "POST", `/orgs/${org.id}/workspaces/${ws.id}/documents`, { title:
    "Page" })).json());
    expect((await as(guest, "GET", "/orgs")).status).toBe(200); // the guest exists (the dev header makes them on first sight)
    const share = Member.parse(await (await as(owner, "PUT", `/documents/${doc.id}/shares`, { email: guest, role: "editor"
    })).json());

    SessionResponse.parse(await (await as(guest, "POST", `/documents/${doc.id}/session`)).json());
    expect((await as(owner, "DELETE", `/documents/${doc.id}/shares/${share.userId}`)).status).toBe(204);
    expect((await as(guest, "POST", `/documents/${doc.id}/session`)).status).toBe(404); // revoked: counted all the same
    await expectLimited(await as(guest, "POST", `/documents/${doc.id}/session`));
    await expectLimited(await as(guest, "POST", `/documents/${doc.id}/session`)); // refusals do not stretch the wait (the window's end)

    // The owner mints on their own count (their sixth request, under the per-user eight).
    SessionResponse.parse(await (await as(owner, "POST", `/documents/${doc.id}/session`)).json());
  });
});

```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>new BlockList()</code> with <code>addSubnet</code> and <code>check</code> : CIDR matching from <code>node:net</code>	<code>IpAddressMatcher</code> from Spring Security	<code>ipaddress.ip_network(...)</code> and <code>in</code>
<code>isIP(address) : 0, 4 or 6</code>	<code>InetAddress.getByName</code> (which may resolve a name: do not)	<code>ipaddress.ip_address</code> raising
the walk from <code>hops.length - 1</code> down	Tomcat's <code>RemoteIpValve</code> does the same walk with <code>internalProxies</code>	unicorn's <code>forwarded_allow_ips</code> gates the header; the walk is yours
<code>Record<string, readonly (readonly [string, number])[]> : named sets of subnets</code>	a <code>Map<String, List<Subnet>></code>	a dict of lists of tuples
<code>z.string().optional().transform((value, ctx) => { ... ctx.addIssue(...); return z.NEVER }) : a parse that can refuse</code>	a <code>@ConfigurationProperties</code> validator	a pydantic <code>field_validator</code> raising
<code>hono.request(path, init, { incoming: { socket: { remoteAddress } } }) : a peer address for an in-process request</code>	<code>MockHttpServletRequest.setRemoteAddr</code>	Django's <code>RequestFactory(REMOTE_ADDR=...)</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Believing X-Forwarded-For from anyone was rejected, and so was the leftmost entry.

Both are the default in more frameworks than one would like. With either, a per-address limit is a per-*claimed*-address limit, and a guesser with a for loop has infinite addresses. The integration test sends four requests over a real socket with four different headers to an api that trusts nobody and asks for `[200, 200, 200, 429]` : the header changed and the count did not.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Postgres for a per-request write, with its ceiling named.

One upsert per request and a hot row per user is a real cost: a few thousand requests a second on one primary is the ceiling the commit names. It was accepted because the count must be shared across instances either way, `Db.take` exists and is proven, every counted route already reads Postgres for identity, and the failure mode is closed for free. The upgrade, Redis `INCR` failing open, is one function behind the same seam.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The probes exempted after asking what an orchestrator does with a 429.

A liveness probe that is refused is a container that is restarted; a readiness probe that is refused is a pod taken out of the load balancer. A limit that can do either under load makes the outage it was meant to prevent. The webhook is exempt for a different reason: its gate is the HMAC, and a limited delivery only delays the reconcile.

The principle: a rate limit is only as good as the identity it counts. Count the user when you know one; when you do not, count an address you can actually attribute, which means the TCP peer or an `X-Forwarded-For` entry a proxy you configured has vouched for, read from the right. And exempt the two callers whose refusal would hurt you more than their traffic: the probes and the signed webhook.

8 · Trade-offs and comparisons: where this design sits

The questions an interviewer asks about job hardening, with the honest answers for this one.

QUESTION	THIS REPO	JAVA WORLD	PYTHON WORLD
Duplicate presses	an <code>Idempotency-Key</code> claimed by a primary key in the job's transaction; a replay reads the job's row; 422 on a different request; 24 h	Stripe's contract, hand-built: Spring has no built-in; Spring Integration's <code>IdempotentReceiver</code> dedupes messages, not HTTP	Django: hand-built, or <code>django-idempotency-key</code> (cache-backed, so not transactional)
A dead worker	a 1 s heartbeat on the row, a 15 s sweep in one <code>skip locked</code> statement, 3 attempts, <code>worker_lost</code>	Spring Batch: restartable steps from the <code>JobRepository</code> , by hand; Quartz misfires; Kafka consumers: a rebalance re-delivers the partition	Celery: <code>acks_late</code> and a visibility timeout; the task runs again and may run twice
A slow worker mistaken for dead	the attempt fences <code>finish</code> , <code>heartbeat</code> and <code>report</code> ; the slow one learns <code>lost</code> and aborts	optimistic locking with <code>@Version</code> is the same fence if you remember to use it	the same, by hand
A replayed step	ids as hashes of (job, step, op); the replica dedupes nodes, the room dedupes ops	Kafka's idempotent producer (sequence per partition); exactly-once semantics with transactions	the task is asked to be idempotent
Redis lost	nothing to do: the sweep rebuilds the queue from Postgres, the lease floor comes from the journal; proven by a chaos script	Sidekiq/Resque on Redis: the queue is the truth and a flush is loss; RabbitMQ with durable queues survives a restart, not a purge	Celery with Redis: the same loss; with RabbitMQ durable queues, a restart
Progress	the steps on the row, the last 50, fenced by the attempt; the 1 s status poll carries them; a reload asks for the newest run	Spring Batch's <code>StepExecution</code> counts; a UI polls	Celery's <code>update_state(meta=...)</code> to the result backend: the same idea, in Redis
Per-tenant limits	a fixed window in Postgres, in the run's transaction; <code>Retry-After</code> from the database's clock; cannot fail open	Bucket4j with a JDBC or Redis backend; resilience4j per instance	django-ratelimit (cache), or a Redis <code>INCR</code>
Per-request limits	the same upsert per user, per address, per mint, per email; probes and webhook exempt	Spring Cloud Gateway's <code>RequestRateLimiter</code> (Redis, token bucket); nginx <code>limit_req</code> in front	nginx <code>limit_req</code> ; django-ratelimit per view
Client address	the rightmost <code>X-Forwarded-For</code> entry past a trusted proxy; a typo in <code>TRUST_PROXY</code> refuses to start	<code>RemoteIpValve</code> / <code>ForwardedHeaderFilter</code> with trusted proxies configured	gunicorn <code>forwarded_allow_ips</code> ; django-ipware with <code>proxy_count</code>

What is deliberately simple. Fixed windows and constants for the limits (an env var for the AI limit, a table of four rules for HTTP), one limit for every org, IPv6 addresses keyed whole, `rate_limits` rows never pruned (one per user, address and email), idempotency keys swept only when their user next sends one, a step that may show a second late, fifty steps kept, a replay that enqueues again, a retried AI attempt whose different plan adds different nodes. Every one is named where it lives, with its ceiling and its upgrade.

What is not solved. A `git_events` row left `running` by a killed git peer stays there: resetting it is easy, but replaying a half-applied push would refuse it whole as `reused_node_id`, so push-ops must become replay-safe first. A retried ship that finds nothing new to commit overwrites its output with `commit: null`, and if the git peer had not yet handled the first attempt's push, that push would be diffed. Usage for runs that fail or exhaust their budget, and per-turn token accounting, are the E3.4 review's items 1 and 2 and were outside this epic's paths. The sync side's byte and fan-out budgets are the bead's deferred notes.

The interview answer in one breath: an idempotency key claimed by a primary key inside the job's own transaction, so twenty simultaneous retries block on one row and answer from the job as it now is; a heartbeat on the row, a stale sweep in one `skip locked` statement, and the attempt number in every write as the fence that makes a false "dead" harmless; ids that are hashes of job, step and content so a replay is deduplicated by the replica and the room; Redis as a cache whose loss the sweep and the fence floor repair, proven by a chaos script with vacuity guards; progress as a fenced column read by the poll that already existed; and rate limits as one upsert in Postgres, inside the run's transaction for AI runs and per request for everything else, keyed by the user or by an address a trusted proxy vouched for, with the wait computed from the database's clock and no way to fail open.

9 · Testing a crash you cannot schedule, a race you can, and a limit to the second

The split from the earlier lessons holds: every decision has a pure test that ran on every commit; every I/O rule has an integration or chaos check that is written and unrun. Six shapes are new to this epic.

Twenty at once. `Promise.all` over twenty `fetch` es is the whole concurrency harness for the idempotency and the rate-limit tests. It works because the claim is serialised by the database, not by the test: the assertion is on rows (`select id, status from jobs`) and on the set of ids in the answers, never on timing. Two api instances over one database (`buildApp` twice) prove the count is shared.

The dead worker, simulated by moving a timestamp. `kill -9` cannot be scheduled inside a vitest process, but what it leaves can: a row with `status = 'running'` and a `heartbeat_at` in the past. `silentFor(jobId, 60)` is one `UPDATE`, and the whole sweep can then be asserted in milliseconds. The slow-worker case is the same trick the other way: bump `attempts` and the beat by hand, as the sweep and another claim would, while the first worker's handler is still open, and watch its finish land nowhere.

FROM `apps/worker/src/crash.int.test.ts`

```
// The wipe: every key of this file's prefix, as FLUSHALL would (the test Redis is shared, so not FLUSHALL itself).
// One script, so it is atomic as FLUSHALL is: nothing lands between listing the keys and deleting them.
const redis = await inspect.client;
redis.defineCommand("wipe", { numberOfKeys: 0, lua: "local keys = redis.call('KEYS', ARGV[1]) for _, key in
ipairs(keys) do redis.call('DEL', key) end return #keys" });
expect(await redis.runCommand("wipe", [`${prefix}:*`])).toBeGreaterThan(0);
```

An atomic wipe of one prefix. The test Redis is shared, so `FLUSHALL` would break the other suites; a `KEYS` then a loop of `DEL` s from the client would let a key land in between. One Lua script does both in Redis's single thread, which is exactly how `FLUSHALL` is atomic, and `defineCommand` is ioredis's way to add one.

FROM `apps/api/src/rate-limit.int.test.ts`

```
/** Seconds to the end of the current window by the DATABASE's clock (the one the limiter reads). */
const secondsLeft = async (windowSeconds: number): Promise<number> =>
  Number(((await ctx.db.rawQuery("select ($1::float8 - mod(extract(epoch from clock_timestamp()),
$1::float8::numeric)::float8) as remaining", [windowSeconds])) as { rows: { remaining: number }[] }).rows[0]?.remaining);
```

The wait, to the second, by the right clock. A test that computed the expected `Retry-After` from `Date.now()` would be comparing two clocks; `secondsLeft` asks Postgres, the clock the limiter reads, and allows two seconds for the round trip and the rounding. The key-not-used-up test then *sleeps exactly what the header said* and expects 201: the assertion is that the promise the header makes is kept.

FROM `apps/api/src/http-limits.test.ts`

```
function app({ trust = "loopback" } = {}) {
  const hits = new Map<string, number>();
  const charged: string[] = [];
  const lookups: string[] = [];
  const db = {
    take: (key: string, rule: Rule) => {
      charged.push(key);
      const n = (hits.get(key) ?? 0) + 1;
      hits.set(key, n);
      return Promise.resolve(n <= rule.limit ? { ok: true } : { ok: false, retryAfterSeconds: 7 });
    },
    upsertUser: ({ email, name }: { email: string; name: string }) => Promise.resolve({ id: `id-${email}`, email, name }),
    listOrgsFor: () => Promise.resolve({ items: [], nextCursor: null }),
  }
```

```

    getDocumentForMember: (id: string) => { lookups.push(id); return Promise.resolve(undefined); },
    ping: () => Promise.resolve(),
  };
  const hono = buildApp({ db: db as unknown as Db, identify: devHeaderIdentity, sessions: SESSIONS, enqueue: () =>
    Promise.resolve(), limits, trustProxy: trustedProxies(trust), webhookSecret: "w".repeat(32), allowAttempt: () =>
    Promise.resolve({ ok: true }) });
  /** `peer`: the socket's address, as @hono/node-server hands it over. */
  const call = (path: string, { method = "GET", headers = {}, peer }: { method?: string; headers?: Record<string, string>;
    peer?: string } = {}) =>
    hono.request(path, { method, headers }, peer === undefined ? undefined : { incoming: { socket: { remoteAddress: peer }
    } });
  return { call, charged, lookups };
}

```

The wiring against an in-memory limiter. Which key each request is charged to, and which requests are charged at all, is a question about the middleware, not about Postgres; `http-limits.test` gives `buildApp` a `take` that records keys and counts in a `Map`, and asserts the exact list of keys per request sequence. Hono's third argument to `request` plants a peer address, so the client-address rules run in-process. The commit records the mutation check: dropping the probe exemption or the mint charge turns the test red.

FROM `scripts/chaos/rebuild.test.ts`

```

describe("runViolations", () => {
  it("passes runs that each ran once and journaled every step once", () => {
    expect(runViolations({ steps, runs: [done("a", 3), done("b", 2)], waitingAtWipe: 1, runningAtWipe: 1 })).toEqual([]);
  });

  it("reports a run that never ended, or a run claimed twice", () => {
    expect(runViolations({ steps, runs: [{ ...done("a", 3), status: "queued" }, { ...done("b", 2), attempts: 2 }],
    waitingAtWipe: 1, runningAtWipe: 1 })).toEqual([
      "run a ended queued, not succeeded",
      "run b was claimed 2 times: the wipe started it again",
    ]);
  });

  it("reports a missing run, a duplicate op or node, and a seq gap", () => {
    expect(runViolations({ steps, runs: [{ ...done("b", 2), ops: 3, seqs: 2, maxSeq: 3 }], waitingAtWipe: 1,
    runningAtWipe: 1 })).toEqual([
      "run a has no row",
      "run b journaled ops=3 opIds=2 nodes=2, not 2 of each",
      "run b's document has 2 distinct seqs up to 3: a gap or a repeat",
    ]);
  });

  it("says a run with nothing waiting or nothing running at the wipe proved nothing", () => {
    expect(runViolations({ steps, runs: [done("a", 3), done("b", 2)], waitingAtWipe: 0, runningAtWipe: 0 })).toEqual([
      "vacuous: no job was waiting in Redis at the wipe, so no queue had to be rebuilt",
      "vacuous: no job was running at the wipe",
    ]);
  });
});

```

The chaos judge as a pure function. A chaos script is a long, slow, Docker-bound thing; its *verdict* need not be. `rebuild.ts` takes what the script read afterwards and returns a list of violations, and `rebuild.test.ts` feeds it a claimed-twice run, a seq gap, a missing run and an empty wipe, and asks for the exact sentences. That is where the vacuity guards were mutation-checked, without running Docker once.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>Promise.all(Array.from({ length: 20 }, () => fetch(...)))</code>	<code>CompletableFuture.allOf</code>	<code>asyncio.gather</code>
<code>update jobs set heartbeat_at = now() - make_interval(secs => \$2) : the past, as a fixture</code>	the same SQL from the test	the same
<code>redis.defineCommand("wipe", { numberOfKeys: 0, lua {} })</code>	Jedis <code>eval ; Lettuce eval</code>	<code>redis.register_script</code>
<code>expect(Math.abs(retry - left)).toBeLessThanOrEqual(2) : a tolerance, stated</code>	<code>assertEquals(expected, actual, delta)</code>	<code>pytest.approx(left, abs=2)</code>
<code>const hits = new Map<string, number>() as the whole limiter</code>	a <code>ConcurrentHashMap</code> fake	a <code>dict</code>
<code>describe(...) + it(...)</code> over exact violation strings	<code>assertIterableEquals</code>	<code>assert violations == [...]</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The integration tests stand in for the chaos scripts, not the other way round.

`crash.int` and `job-heartbeat.int` cover the sweep, the fence and the wipe in seconds, with the crash simulated by a timestamp; `kill-worker-resumes` and `redis-wipe-rebuild` do it to the real compose stack with a real `SIGKILL` and a real `FLUSHALL`, in minutes. Both exist because they answer different questions: the first, "is the logic right"; the second, "is the wiring right", including the stub worker running inside the worker image with the compose environment.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A timing assertion on the retry, by Postgres's clock.

The chaos script asserts that attempt 2 started at least `staleMs` after attempt 1's last heartbeat. Both timestamps are read from the row in one `psql` call and compared with each other, never with the host's clock: a container's clock and the host's may disagree, and a test that compared them would be flaky in exactly the way that hides the bug it looks for.

Honest status, repeated here because it matters: the unit tests (`replay.test`, `progress.test` in the worker and the web, `limit.test`, `usage.test`, `client-address.test`, `http-limits.test`, `rebuild.test`, the contracts, the config, the route table in `app.test`) ran green on every commit. The integration suites (`idempotency.int`, `crash.int` including the wipe, `job-heartbeat.int`, `starvation.int`, `run-progress.int`, `rate-limit.int`, `usage.int`, `http-rate-limit.int` with the mint limit), the two end-to-end specs (`progress`, `usage-view`) and the two chaos scripts (`kill-worker-resumes`, `redis-wipe-rebuild`) are written, reviewed and committed, and have not been run. Section 10 is what running them looks like.

10 · By hand: press twice, kill the worker, flush Redis, reload mid-run, hit the limit

The exact commands and what each should show. The api is on host port 3000, the web app on 5173; Redis and Postgres are reached through compose. The dev header works in this development stack, which is what lets one terminal act as a person. `DOC` is a document id from the canvas's `?doc=`.

0 · BRING THE STACK UP

```
./init.sh                                # migrate runs 0018 (idempotency_keys), 0019 (heartbeat_at, attempts), 0020
(rate_limits)
open http://localhost:5173/              # sign in, New document; note ?doc=<id>
export DOC=<id> H='x-dev-user: you@example.com' J='content-type: application/json'
```

1 · ONE KEY, ONE JOB (F27)

```
K=$(uuidgen)
curl -s -H "$H" -H "$J" -H "idempotency-key: $K" -d '{"instruction":"add a hero"}' localhost:3000/documents/$DOC/runs | jq
'{id,status}' # 201
curl -s -H "$H" -H "$J" -H "idempotency-key: $K" -d '{"instruction":"add a hero"}' localhost:3000/documents/$DOC/runs | jq
'{id,status}' # 201, the SAME id, status as it is now
curl -si -H "$H" -H "$J" -H "idempotency-key: $K" -d '{"instruction":"add a footer"}' localhost:3000/documents/$DOC/runs |
head -1 # 422 idempotency_key_reused
for i in $(seq 1 10); do curl -s -o /dev/null -w '%{http_code}' -H "$H" -H "$J" -H "idempotency-key: $(uuidgen)" -d
'{"instruction":"x"}' localhost:3000/documents/$DOC/runs & done; wait # one 201 (or none: busy), the rest 409: one
unfinished run per document
docker compose exec -T postgres psql -U noon -d noon -c "select key, request->'queue' as queue, job_id from
idempotency_keys" # the claimed keys, each naming its job; the 409s left none
```

2 · KILL THE WORKER MID-RUN (F28)

```
docker compose exec -T postgres psql -U noon -d noon -c "select id, status, attempts, heartbeat_at from jobs where queue =
'ai' order by created_at desc limit 1" # running, attempts 1, the beat moving every second
docker compose kill -s SIGKILL worker # nobody says anything
watch -n1 'docker compose exec -T postgres psql -U noon -d noon -tAc "select status, attempts, now() - heartbeat_at from
jobs where queue = ''ai'' order by created_at desc limit 1"'
# running|1|the age growing... past 15 s: worker-ship's or worker-sandbox's
sweep requeues it: queued|1|; they do not handle ai, so it waits
docker compose start worker # its first sweep offers the row; claim: running|2|; the run replays its
early steps (nothing added twice) and ends succeeded|2
docker compose exec -T postgres psql -U noon -d noon -c "select count(*), count(distinct op_id), count(distinct op-
->'nodeId') from op_journal where document_id = '$DOC' and actor_kind = 'agent'" # three equal numbers
```

3 · FLUSH REDIS UNDER A WAITING RUN (F28, §4)

```
docker compose exec -T redis redis-cli --no-auth-warning LLEN bull:ai:wait # start a run on a second
document while one runs: 1 waiting
docker compose exec -T redis redis-cli --no-auth-warning GET lease:$DOC # <token>:<node>: the room's
owner
docker compose exec -T redis redis-cli --no-auth-warning FLUSHALL # OK: every key gone, every
connection open
docker compose exec -T redis redis-cli --no-auth-warning LLEN bull:ai:wait # 0, then within 5 s (the
sweep) 1 again: offered from Postgres
docker compose exec -T postgres psql -U noon -d noon -c "select status, attempts from jobs where queue = 'ai' order by
created_at desc limit 2" # both end succeeded, attempts 1: nothing ran twice
docker compose exec -T redis redis-cli --no-auth-warning GET lease:$DOC # a LARGER token; the canvas
blinked once (4409) and is live again
docker compose exec -T postgres psql -U noon -d noon -c "select fence_token from documents where id = '$DOC'" # the same
token: one owner
```

4 · PROGRESS, AND A RELOAD MID-RUN (F30)

```
open http://localhost:5173/?doc=$DOC # Ask the AI for 'a card with 8 buttons'; the list under the panel grows:
Added Card, Added Button...
# reload the page while it runs: the same run, the same steps, still running,
Cancel still there; after the end, reload again: the list stays
curl -s -H "$H" localhost:3000/documents/$DOC/run | jq '.run.steps' # [{tool, ok, detail}...]: the model's argument as
text, at most 80 characters
docker compose exec -T postgres psql -U noon -d noon -c "select jsonb_array_length(output->'steps') from jobs where queue
= 'ai' order by created_at desc limit 1"
```

5 · THE LIMITS (F31)

```
AI_RUNS_PER_HOUR=2 docker compose up -d api # or edit .env; the default is 60
for d in $DOC $DOC2 $DOC3; do curl -si -H "$H" -H "$J" -d '{"instruction":"x"}' localhost:3000/documents/$d/runs | grep -i
'^HTTP\|retry-after'; done # 201, 201, 429 with Retry-After: seconds to the top of the hour
curl -s -H "$H" -H "$J" -d '{"instruction":"x"}' localhost:3000/documents/$DOC3/runs #
{"error":"rate_limited","retryAfterSeconds":N}: the same number as the header
for i in $(seq 1 305); do curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/auth/me; done | sort | uniq -c # 300 x
200, 5 x 429: per address, nobody signed in
curl -si -H 'x-forwarded-for: 203.0.113.9' localhost:3000/auth/me | head -1 # 200: from the host, the peer is Docker's
gateway (private, trusted in compose), so the header is believed and this is a fresh address
docker compose exec -T postgres psql -U noon -d noon -c "select key, win, hits from rate_limits order by hits desc limit
5" # address:..., user:..., ai_run:<org>
```

```
for i in 1 2 3; do curl -s -o /dev/null -w '%{http_code}\n' -H "$J" -d '{"email":"you@example.com","password":"wrong"}'
localhost:3000/auth/signin; done # 401 x 10, then 429 too_many_attempts with Retry-After (run it eleven times)
```

6 · THEN THE CHECKS THAT PROVE IT WITHOUT HANDS

```
pnpm exec vitest run apps/worker/src/replay.test.ts apps/worker/src/progress.test.ts apps/web/src/progress.test.ts
packages/db/src/limit.test.ts apps/api/src/client-address.test.ts apps/api/src/http-limits.test.ts
scripts/chaos/rebuild.test.ts # the pure halves
pnpm exec vitest run --config vitest.integration.config.ts apps/api/src/idempotency.int.test.ts
apps/worker/src/crash.int.test.ts packages/db/src/job-heartbeat.int.test.ts \
  apps/worker/src/starvation.int.test.ts packages/db/src/run-progress.int.test.ts apps/api/src/rate-limit.int.test.ts
packages/db/src/usage.int.test.ts apps/api/src/http-rate-limit.int.test.ts
pnpm exec playwright test e2e/progress.spec.ts e2e/usage-view.spec.ts
node scripts/chaos/kill-worker-resumes.ts && node scripts/chaos/redis-wipe-rebuild.ts # against the real stack; each
prints one JSON line, exit 0 = PASS
make check # the whole gate, as the pre-commit hook runs it
```

What to do when a step does not match this page: the page was cut from the code at commit `1c9db7c`, and the code is right until a test says otherwise. A step that fails is a finding: write it as *sequence* → *false belief* → *consequence* → *smallest fix* (SPEC §4a's form) and file it against the bead that owns the code.

11 · What the type system could not see, epic 9 edition

Everything below compiled under `strict`, passed the linter and the first tests, or was rejected at design time by asking what the types could not ask.

WHAT WENT WRONG, OR WOULD HAVE	WHY	FIX
twenty retries making twenty jobs	each request is a function call	a key claimed by a primary key; the loser blocks on the row
a look-then-insert for the key	<code>get_or_create</code> reads fine	<code>on conflict do nothing</code> : one statement (drill 2)
the key as columns on the job	one table is simpler	a joined ship has no row to hold it; a table of its own
a malformed key silently ignored	"optional" reads as "skip if bad"	400: the retry it protects would not be protected
a replay answered from the first response	the same request, the same answer	the job's row as it now is
a refused run using up its key	record every outcome	roll the claim back with the run
a key resolving to another tenant's job	a string is a string	<code>(org_id, user_id, key)</code>
<code>kill -9</code> leaving <code>running</code> for ever	a promise that never settles has no type	a heartbeat on the row, a sweep
a slow worker finishing over the next attempt	two <code>UPDATE</code> s look alike	<code>and attempts = \$n</code> in every write
a beat that only beats	one timer, one job	it also answers cancel and <code>lost</code>
two sweeps taking one row	one statement each	<code>for update skip locked</code>
two stale ships requeued for one document	each row on its own	<code>distinct on</code> ; 0011 refused the pair
a cancelled job run again by the sweep	stale is stale	<code>cancel_requested_at</code> ends it as <code>cancelled</code>
a dead sandbox's address kept	output is output	cleared on requeue; the next attempt reports one
a row from before 0019 never judged stale	<code>heartbeat_at</code> is null	<code>coalesce(heartbeat_at, started_at)</code>
a replayed <code>add_node</code> adding twice	a new call, a new id	ids as functions of (job, step)
an op id of (job, step) alone	an id is an id	the op's content in the hash: a resend for ever otherwise
a replay refused instead of done	<code>duplicate_node</code> is a refusal	same component under the same parent: done
Redis flushed under a running job	the lock is gone	nothing: its truth was never in Redis
a chaos round that wiped an empty queue	passed	vacuity guards: <code>LLEN</code> before the wipe
steps from a dead attempt shown	output is output	<code>report</code> fenced; cleared on requeue
a later list landing before an earlier	two awaits	a promise chain; awaited before <code>finish</code>
a bidi override in a step's detail	a string	<code>\p{C}</code> stripped; text nodes
an older run replacing the one just started	<code>setRun(latest)</code>	<code>setRun((current) => current ?? latest)</code>
a row that does not parse throwing on read	our own worker wrote it	<code>safeParse</code> ; no steps
N instances allowing $N \times$ the limit	a counter is a counter	the count in Postgres
a refused run counted	count then insert	the hit inside the run's transaction
a late hit resetting the window	<code>win = excluded.win</code>	<code>greatest</code> and <code>>=</code>

<code>now()</code> mid-transaction	it is "now"	<code>clock_timestamp()</code>
a wait of 0 seconds	rounding down	<code>Math.max(1, Math.ceil(...))</code>
the limiter failing open when Redis is away	"allow" felt kind	no Redis: Postgres or nothing
the probes rate limited	a route is a route	never counted
a 401 not counted	nobody to charge	charged to the address; then 429
<code>X-Forwarded-For</code> believed from anyone	a header is a header	only past a trusted peer
the leftmost entry as the client	"the original IP"	the rightmost entry a trusted hop vouched for
<code>::ffff:127.0.0.1</code> not loopback	two strings	<code>unmap</code>
a typo in <code>TRUST_PROXY</code> trusting no one	an empty list	refuse to start
a revoked guest's mints costing a lookup each	404 is cheap	<code>mint:</code> charged before the lookup
an email too long for the key column	a string is a string	its SHA-256

The pattern, extended once more: lesson 8 said types cannot see who is asking or what a value costs. Epic 9 adds the two that make distributed jobs hard: types cannot see *how many times* a call happens (a function called twice has the same signature as one called once, and a message delivered twice the same shape), and cannot see *which clock* a number came from (the host's, the container's, the database's, all `number`). Both were answered as before: put the truth in one place and let the database's own uniqueness and locks do the counting; make every write name the attempt or the claim it belongs to; take the clock from where the truth is; and write a test that provokes the exact case: twenty at once, a timestamp moved into the past, a Lua wipe, a sleep of exactly the promised seconds.

Your turn: three drills

Do these alone. They live in `drills/lesson-9/` and both coded drills start RED; `sh drills/lesson-9/check.sh` shows your progress. Neither needs Docker.

D1 · Extend a type, end to end. A run taken over after its worker died looks, on the panel, exactly like a first attempt: "The AI is working" the whole time, and a bug report cannot tell attempt 1 from attempt 3. Carry the claim count from the row to the screen: `Run` gains a *required* `attempt`, a whole number that is at least 0 (`packages/contracts`); `RunRow` in `packages/db/src/index.ts` reads the `attempts` column that `select *` already returns and maps `attempt: r.attempts`; the type checker then names every hand-built `Run` (start with `apps/worker/src/run.int.test.ts`), and that list is the drill; `attemptWords(run)` in `apps/web/src/progress.ts` is "" for attempt 0 or 1 and, for $n \geq 2$, a sentence starting "Attempt n" that says a worker died and another picked the run up, which `AiPanel.tsx` shows under the status sentence as text. make check must stay green.

Predict first: the sweep gives the run back and a worker claims it as attempt 2 while a slow first worker may still be alive. Whose steps does the panel show while both exist, and which `where` clause makes that true?

D2 · Fix a planted bug. `drills/lesson-9/d2-idempotency-key/keys.ts` is `withKey` cut down to its decision (the store, the claim, the answer from the row) with one bug from this chapter planted in it. Its store is in memory with a round trip per call, so twenty presses can arrive together. Read the failing assertion and say, before you open the code, how many jobs twenty presses made and what happened between the moment each request looked for the key and the moment it wrote it. The fix replaces two calls with one.

D3 · Explain a decision before reading mine. Answer out loud or on paper, then open the answer.

▼ 1. The idempotency key is claimed with `insert ... on conflict do nothing` inside the transaction that makes the job, before the job exists; a replay is answered by reading the job's row; a run that was refused as busy rolls the claim back. Defend each against the obvious alternative (a look-then-insert, storing the first response, recording the refusal), and say what Postgres does to the second of two claims at the same moment.

A look-then-insert has an `await` between the look and the insert, and twenty requests that arrive together all look before any inserts: each finds nothing and each makes a job; the unique constraint then fails nineteen of them and the code must catch the error and read again, which is the same statement the design starts from, written the long way. With `on conflict do nothing` the claim and the check are one statement, and the second transaction to insert the same primary key *blocks on the first's uncommitted row* until it commits or rolls back: if it committed, the loser gets `rowCount 0`, reads the winner's row in a statement of its own (a statement started earlier would not see the commit) and answers with the winner's job; if it rolled back, the key is free and the loser becomes the winner. Storing the first response, as Stripe does, would answer `queued` to a retry of a run that is now `running`; the person retrying wants to know where the job is, and the job's row is the truth, so the replay reads it. Recording a refusal under the key is right for a declined payment and wrong for a busy document: the person who got the 409 waits and presses again with the same request, and a recorded 409 would answer 409 for 24 hours; rolling the claim back with the refused run makes the key as if never sent, so the retry after the busy run ends may use it. Scoping the key to `(org, user)` keeps a key from ever resolving to another tenant's job, and storing the request lets the same key with another instruction be refused as 422 rather than silently answered with the wrong job.

▼ 2. A running job heartbeats its row every second; a sweep requeues one silent for 15 s; every write of an attempt carries and `attempts = $n`. Why is a heartbeat not a lease, why does the sweep need the fence, and what does a worker that was only slow experience?

A lease (lesson 7) is a promise with a deadline: the holder knows until when it may act, renews before then, and gives up on its own clock; it is the right tool where the holder must act alone for a while, like a room ordering ops. A heartbeat is a report with no promise: the worker says "alive a second ago" and has no idea whether anyone still believes it; the decision to give the job away is made by whoever reads the row. That is enough where the work can be handed to someone else and the only harm of a wrong guess is a duplicate, provided the duplicate is made harmless, which is the fence's job. The sweep needs it because a timeout cannot tell slow from dead: a 20-second GC pause, a blocked event loop or a partition from Postgres looks exactly like `kill -9` for 15 seconds, and the sweep will requeue a job whose worker is about to wake. Without the fence, that worker's `finish("succeeded")` lands on the row attempt 2 is running, and attempt 2's own finish later updates nothing because the status is no longer `running`. With it, the slow worker's next beat returns no row, the store reads that as `lost`, the worker aborts its handler with a warning, and its finish, its beats and its progress reports all match zero rows: they land nowhere. The sweep itself is one statement with `for update skip locked` so every worker process may run it and two never take the same row, `coalesce(heartbeat_at, started_at)` so rows from before the migration are judged too, and a bound of three claims so a job that kills its worker every time still ends, as `worker_lost`.

▼ 3. The AI run limit is one upsert on a `rate_limits` row, taken inside the run's transaction before the insert, with `Retry-After` computed from `clock_timestamp()`; Redis INCR was rejected. Why each, and what does a fixed window get wrong that a token bucket gets right?

Counting in Postgres makes the count shared across every api instance (a counter per process allows N times the limit) without a second store: with Redis, the code must decide what to do when Redis is away, and both answers are bad under load (open: every org unlimited exactly when the system is struggling; closed: no runs with Postgres healthy). Inside the run's transaction, before the insert, makes the count exactly the runs that exist: a busy document, a vanished one, a replayed key and a refused run are all rolled back with their hit, so nothing needs a compensating decrement, and a refused request's idempotency claim rolls back too, so the retry after `Retry-After` may reuse the key. The upsert is atomic per row, so two hits at one moment take turns, and `greatest(win)` keeps a hit that computed its window before a boundary but reached the row after one computed past it from moving the row back and resetting the count. `clock_timestamp()` rather than `now()` because `now()` is the transaction's start and this is several statements in, and the database's clock rather than the api's so two instances with drifting clocks give one wait; the wait is rounded up and

never under one second so a client that waits exactly what it was told is let in. A fixed window lets a key spend its whole limit in the last second of one window and again in the first of the next, twice the rate for a moment; a token bucket refills at a steady rate and allows a burst only up to the bucket's size, which is the shape most public APIs want, and it costs two columns (level, last refill) and a little arithmetic in the same upsert. For sixty AI runs an hour the boundary burst is a nuisance and not a cost, so the fixed window stayed and the ponytail is named where it lives.

Recall first

Look away. Name the three parts of a retry system and what this epic uses for each. Then say which `X-Forwarded-For` entry is the client and under what condition the header is read at all. Then check.

▼ Check yourself

Detection: a 1 s heartbeat on the row, read by a sweep that calls 15 s of silence stale. Redelivery: the row back to `queued`, offered again in the same sweep, at most three claims, then `worker_lost`. Fencing: the attempt number in every write's `where`, so a worker wrongly given up on writes nothing and learns `lost` on its next beat. The client is the *rightmost* entry that is not itself a trusted proxy, walking from the right and skipping trusted hops; and the header is read only when the TCP peer is a trusted proxy (`TRUST_PROXY` : loopback on the host, loopback and private in compose), otherwise the peer is the client and the header is ignored.

Then test it

1. Twenty requests carrying the same `Idempotency-Key` arrive within a millisecond. What happens?

Nineteen get 409 `run_in_progress`

One inserts the key and makes the job; nineteen block inside their own `insert ... on conflict do nothing` on the winner's uncommitted row, then read its job id in a statement of their own and answer with the job as its row now is

The api takes an advisory lock per key

All twenty make a job and the unique index deletes nineteen

No lock is named: the primary key does it. The test asserts twenty 201s, one row in `jobs`, one id in every answer.

2. The same key arrives with a different instruction. And a request refused as busy: what does its key do?

422 `idempotency_key_reused`: the stored request differs; and a busy refusal rolls the whole transaction back, claim included, so the retry after the run ends may use the key

The new instruction replaces the old and the same job runs it

Both are 400

The key is consumed by the 409, like Stripe records errors

Stripe records refusals because a declined card must stay declined on retry; a busy document is a wait, not a verdict.

3. A worker's event loop blocks for 20 seconds mid-job. The sweep has requeued the job and another worker is running it as attempt 2. The first worker wakes and calls `finish("succeeded")`. What happens?

Postgres raises a serialization error

The row becomes succeeded and attempt 2's later finish is ignored

The sweep notices two workers and kills one

Nothing: the update carries and `attempts = 1` and the row says 2, so zero rows match; its next beat returns no row, the store says `lost`, and its handler is aborted

A timeout cannot tell slow from dead, so every false positive would be a duplicate without the fence; the test bumps `attempts` by hand under an open handler and watches the finish land nowhere.

4. Why does the AI run's op id hash the op's content, not only (job, step)?

So two runs on one document never collide

A retry that does something different at step n would otherwise send a new op under an id the room already journaled; the room would answer with the old op's seq, the replica would never match it, and the client would resend for ever

The room requires content-addressed ids

UUIDs must be random

Two runs never collide anyway: the job id is in the hash. The content makes "the same step, a different action" a different op.

5. Redis is flushed while three runs wait in `bull:ai:wait` and four are running. What brings the three back, and what happens to the four?

The worker's sweep offers every row Postgres still calls `queued` again within 5 s, and `claim()` makes a duplicate offer harmless; the four never notice, because their heartbeat and their finish are Postgres rows

BullMQ's stalled checker re-runs all seven

A sentinel key detects the flush and rebuilds the queue

The three are lost; the four fail as `worker_lost`

No production code changed for this bead; the chaos judge asks for `attempts = 1` on every run and refuses a round that wiped an empty queue.

6. A slow dead attempt reports its steps after attempt 2 has started. What does the panel show?

Nothing until the run ends

Both lists merged in order

Attempt 2's steps only: `report` carries and `attempts` = \$n like every write, and the requeue cleared the dead attempt's output so the retry's list started empty

The dead attempt's steps, then attempt 2's appended

The integration test reports from attempt 1 after the requeue and after attempt 2's claim, and asserts an empty list both times.

7. Why is the AI run limit counted in Postgres inside the run's transaction rather than with Redis `INCR` + `EXPIRE` ?

Redis cannot expire keys reliably

Every instance shares the count with no second store to lose, so it cannot fail open; and a busy, refused or replayed request rolls its hit back with the run, so the count is exactly the runs that exist, with no compensating decrement

Postgres is faster than Redis at counting

The limit must survive a Redis flush

The flush is true too, but incidental: the deciding questions are "what when the store is away" and "how is a refused run uncounted".

8. `X-Forwarded-For: 10.0.0.1, 1.2.3.4, 198.51.100.1` arrives from a loopback peer with `TRUST_PROXY=loopback` . Who is charged?

10.0.0.1: the leftmost is the original client

1.2.3.4: the middle entry is the real one

198.51.100.1: the rightmost entry, appended by the trusted proxy that saw the request come from it; everything left of it was written by the client and may say anything

127.0.0.1: the peer is always the client

From an untrusted peer the header is ignored entirely; the integration test sends four different headers over a real socket to an api that trusts nobody and gets `[200, 200, 200, 429]` .

Next: Lesson 10, after epic 10.